

1.

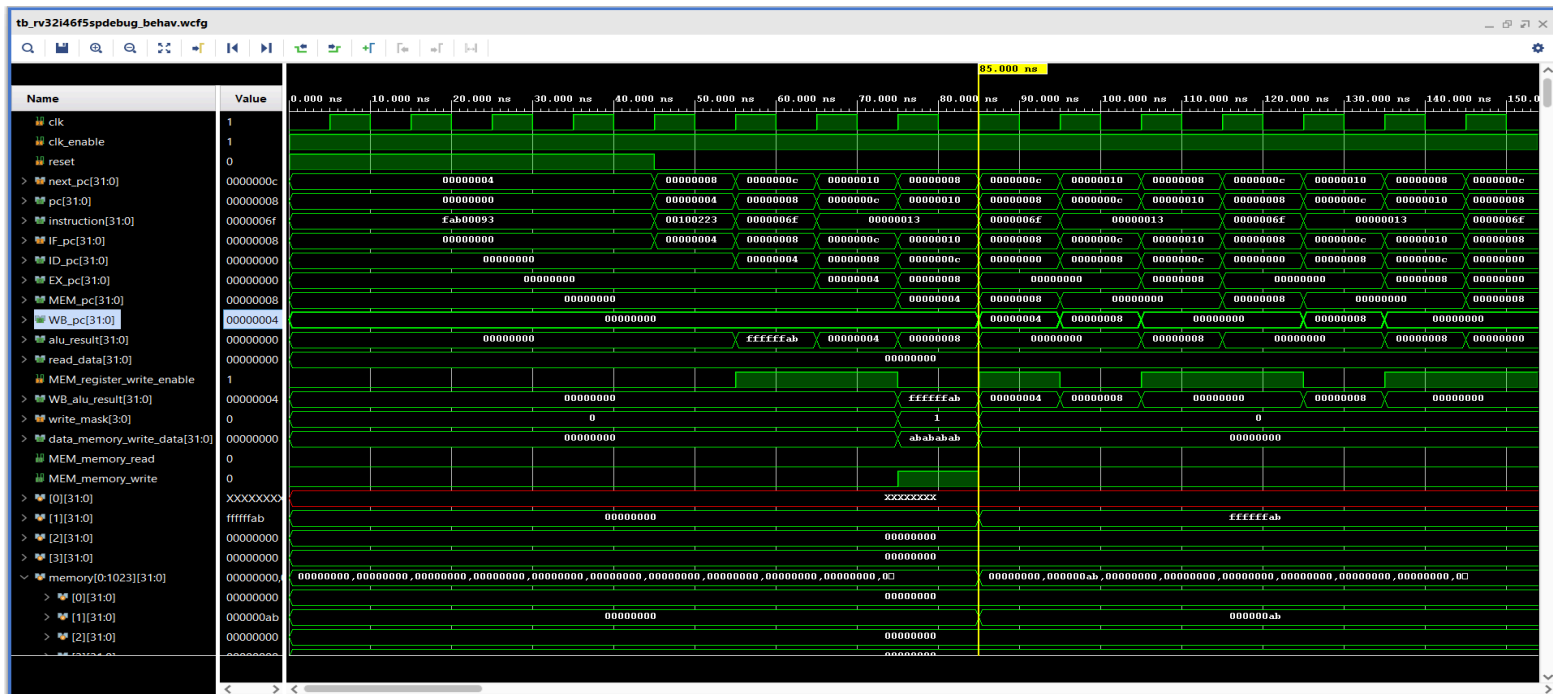
Instruction: sb (store byte)

Type: s-type

Instruction memory:

```
data[0] = {12'hFAB, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // addi x1, x0, 0xFAB
data[1] = {7'd0, 5'd1, 5'd0, `STORE_SB, 5'd4, `OPCODE_STORE}; // sb x1, 4(x0)
data[2] = 32'h0000006f; // jal x0,0 (Infinite loop)
```

Simulation Result:



Comments:

The sb instruction is fetched at PC 0x00000004 and advances through the decode and execute cycles to compute its target memory address. During its fourth cycle in the MEM stage, the write mask is set to 1 to extract the lowest byte from the broadcasted 0xabababab data bus. In its fifth and final cycle, the operation concludes with memory[1] properly updating to 0x000000ab just before retiring in the WB stage.

2.

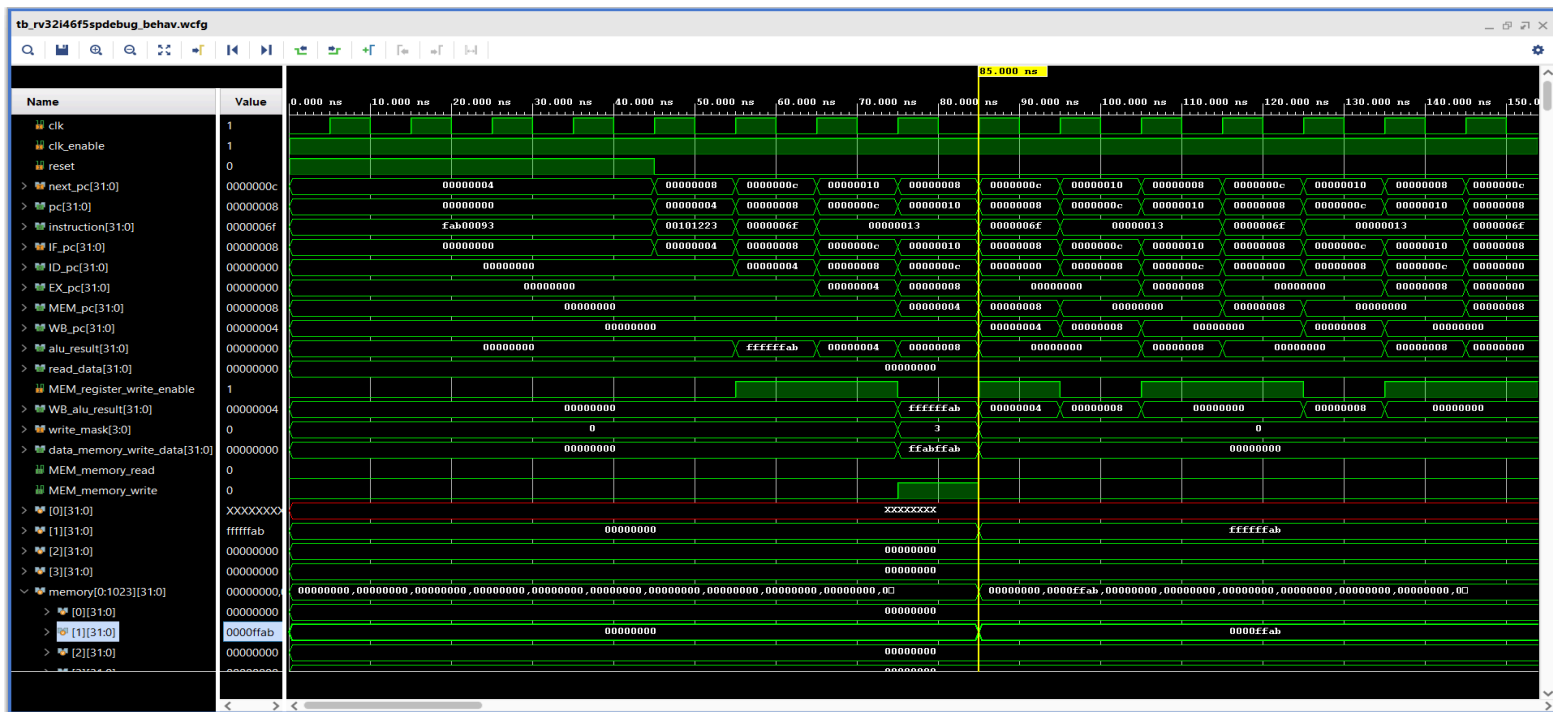
Instruction: sh (store halfword)

Type: s-type

Instruction memory:

```
data[0] = {12'hFAB, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // addi x1, x0, 0xFAB
data[1] = {7'd0, 5'd1, 5'd0, `STORE_SH, 5'd4, `OPCODE_STORE}; // sh x1, 4(x0)
data[2] = 32'h0000006f; // jal x0,0
```

Simulation Result:



Comments:

The sh instruction is fetched at PC 0x00000004 and advances through the decode and execute stages to compute its target memory address. During its fourth cycle in the MEM stage, the write mask is set to 3 to extract the lowest halfword from the broadcasted 0xffabffab data bus. In its fifth and final cycle, the operation safely concludes with memory[1] correctly updating to 0x0000ffab right before retiring in the WB stage.

3.

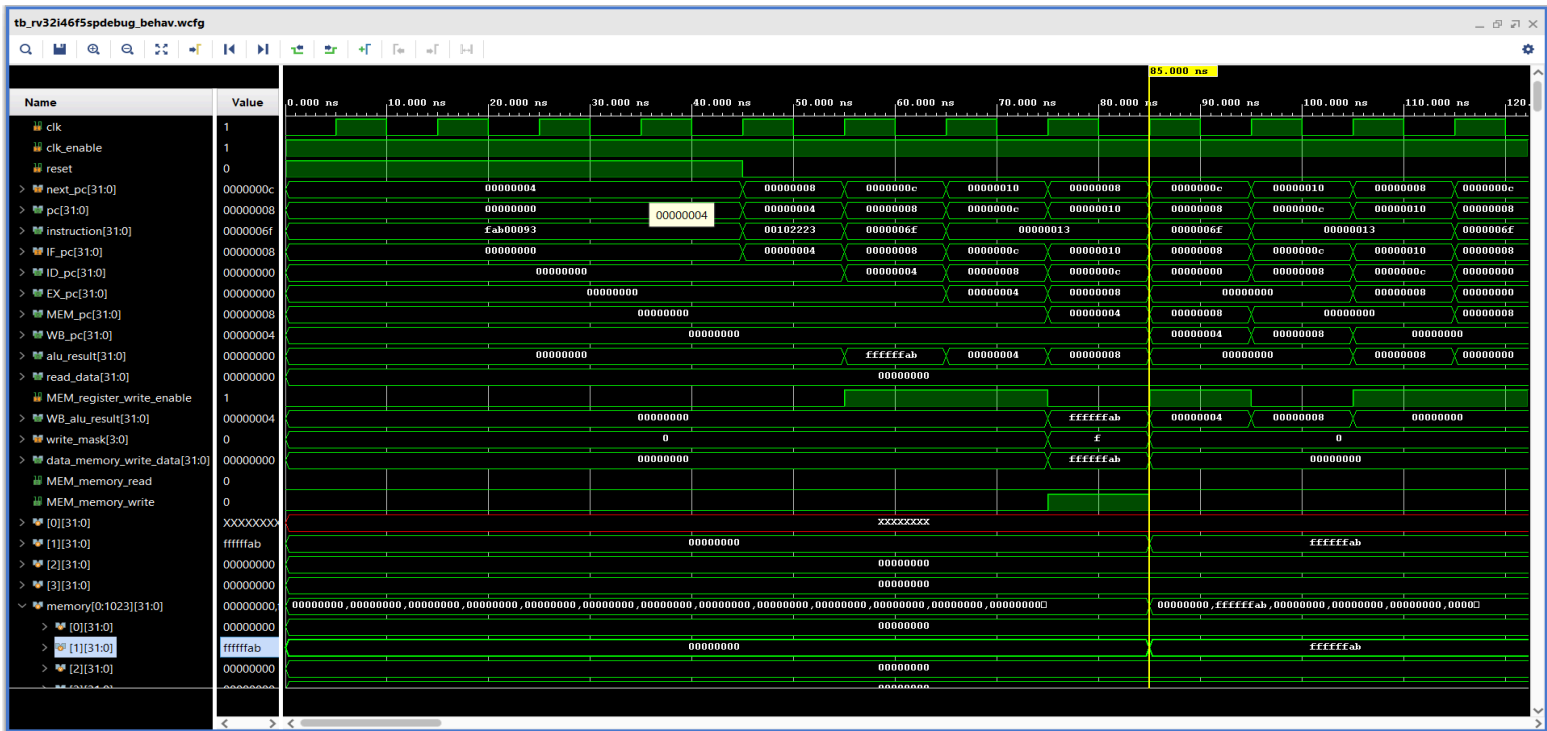
Instruction: sw (store word)

Type: s-type

Instruction memory:

```
data[0] = {12'hFAB, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // addi x1, x0, 0xFAB
data[1] = {7'd0, 5'd1, 5'd0, `STORE_SW, 5'd4, `OPCODE_STORE}; // sw x1, 4(x0)
data[2] = 32'h0000006f; // jal x0,0
```

Simulation Result:



Comments:

The sw instruction is fetched at PC 0x00000004 and advances through the decode and execute cycles to compute its target memory address. During its fourth cycle in the MEM stage, the write mask fully activates as f to pass all 32 bits of the broadcasted 0xfffffab data bus. In its fifth and final cycle, the operation concludes with memory[1] correctly updating to the complete word 0xfffffab just before retiring in the WB stage.

Instruction: lb (load byte)

Type: I-type

Instruction memory:

```
data[0] = {12'h8AB, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // addi x1, x0, 0x8AB (x1 = 0xFFFF8AB)
```

```
data[1] = {7'd0, 5'd1, 5'd0, `STORE_SH, 5'd4, `OPCODE_STORE}; // sh x1, 4(x0)
```

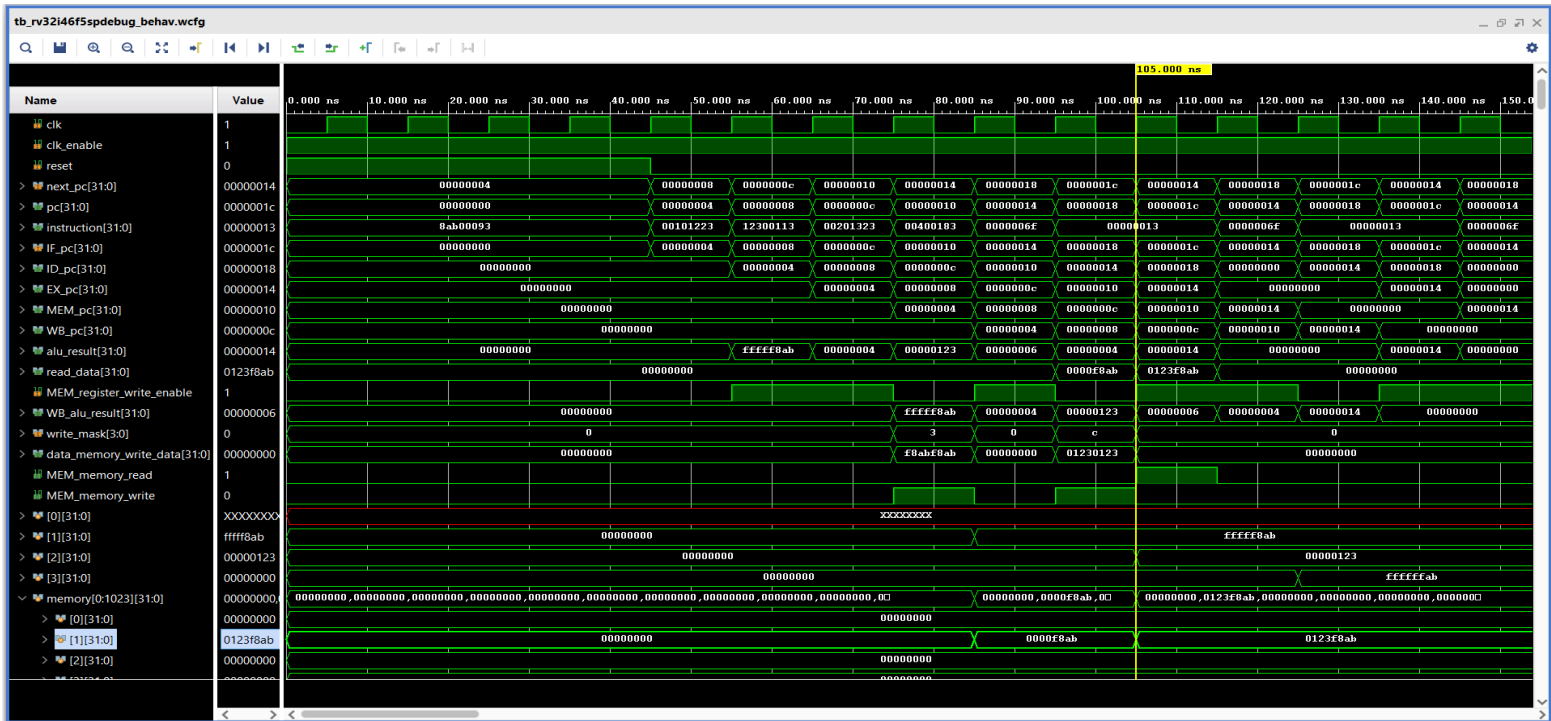
```
data[2] = {12'h123, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE}; // addi x2, x0, 0x123 (x2 = 0x00000123)
```

```
data[3] = {7'd0, 5'd2, 5'd0, `STORE_SH, 5'd6, `OPCODE_STORE}; // sh x2, 6(x0)
```

```
data[4] = {12'd4, 5'd0, `LOAD_LB, 5'd3, `OPCODE_LOAD}; // lb x3, 4(x0)
```

```
data[5] = 32'h0000006f; // jal x0,0
```

Simulation Result:



Comments:

The lb instruction is fetched and advances smoothly through the decode and execute cycles to compute its target memory read address. During its fourth cycle in the MEM stage, the memory module outputs the complete stored word 0x0123f8ab, allowing the processor to precisely extract the lowest byte 0xab. In its fifth and final cycle, the operation successfully concludes by correctly sign-extending this extracted byte to 0xfffffab just before updating the destination register in the WB stage.

5.

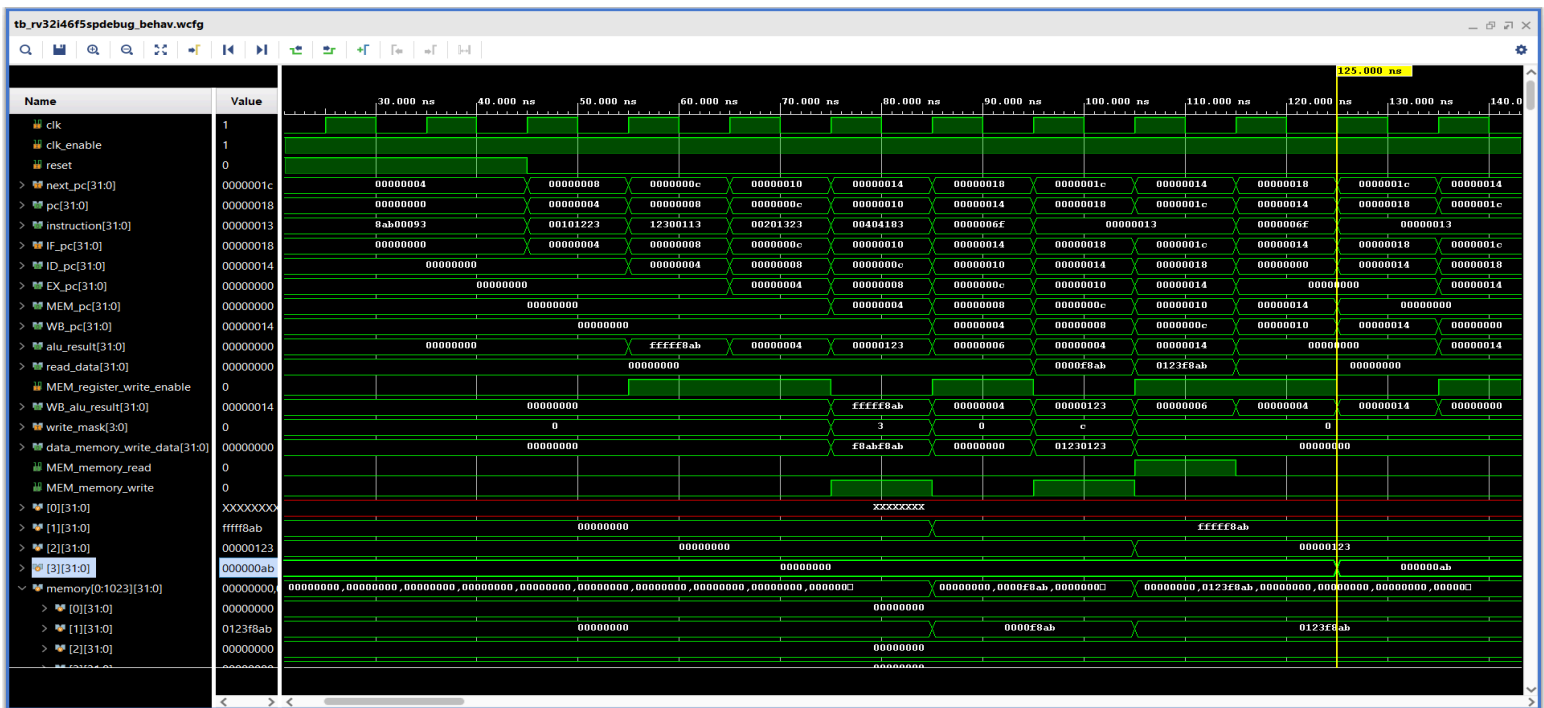
Instruction: lbu (load byte unsigned)

Type: I-type

Instruction memory:

```
data[0] = {12'h8AB, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};  
data[1] = {7'd0, 5'd1, 5'd0, `STORE_SH, 5'd4, `OPCODE_STORE};  
data[2] = {12'h123, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};  
data[3] = {7'd0, 5'd2, 5'd0, `STORE_SH, 5'd6, `OPCODE_STORE};  
data[4] = {12'd4, 5'd0, `LOAD_LBU, 5'd3, `OPCODE_LOAD}; // lbu x3, 4(x0)  
data[5] = 32'h0000006f; // jal x0,0
```

Simulation Result:



Comments:

The lbu instruction is fetched and advances smoothly through the decode and execute cycles to compute its target memory read address. During its fourth cycle in the MEM stage, the memory module outputs the complete stored word 0x0123f8ab, allowing the processor to extract the lowest byte 0xab. In its fifth and final cycle, the operation successfully concludes by correctly zero-extending this extracted byte to 0x000000ab just before updating the destination register in the WB stage.

6.

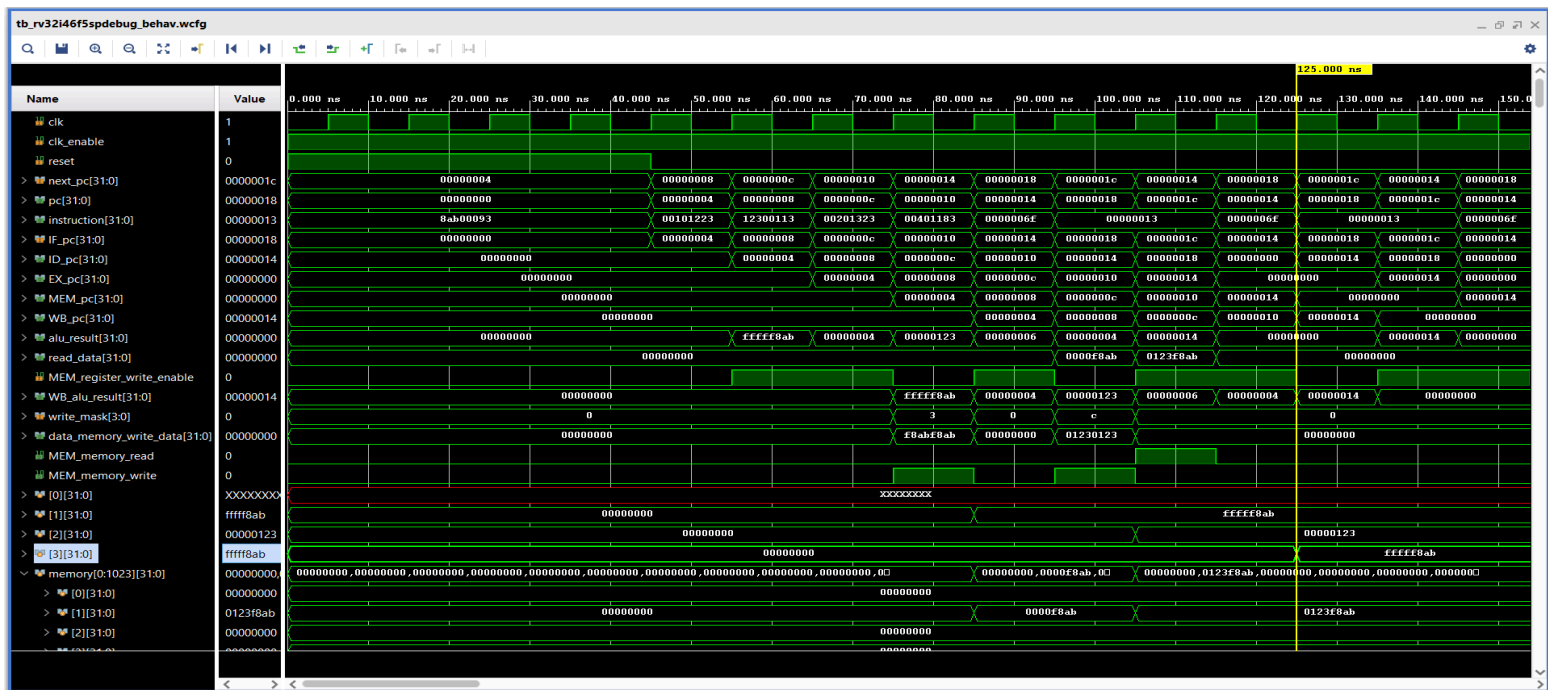
Instruction: lh (load halfword)

Type: I-type

Instruction memory:

```
data[0] = {12'h8AB, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};  
data[1] = {7'd0, 5'd1, 5'd0, `STORE_SH, 5'd4, `OPCODE_STORE};  
data[2] = {12'h123, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};  
data[3] = {7'd0, 5'd2, 5'd0, `STORE_SH, 5'd6, `OPCODE_STORE};  
data[4] = {12'd4, 5'd0, `LOAD_LH, 5'd3, `OPCODE_LOAD}; // lh x3, 4(x0)  
data[5] = 32'h0000006f; // jal x0,0
```

Simulation Result:



Comments:

The lh instruction is fetched and advances smoothly through the decode and execute cycles to compute its target memory read address. During its fourth cycle in the MEM stage, the memory module outputs the complete stored word 0x0123f8ab, allowing the processor to extract the lowest halfword 0xf8ab. In its fifth and final cycle, the operation successfully concludes by correctly sign-extending this extracted halfword to 0xfffff8ab just before updating the destination register in the WB stage.

7.

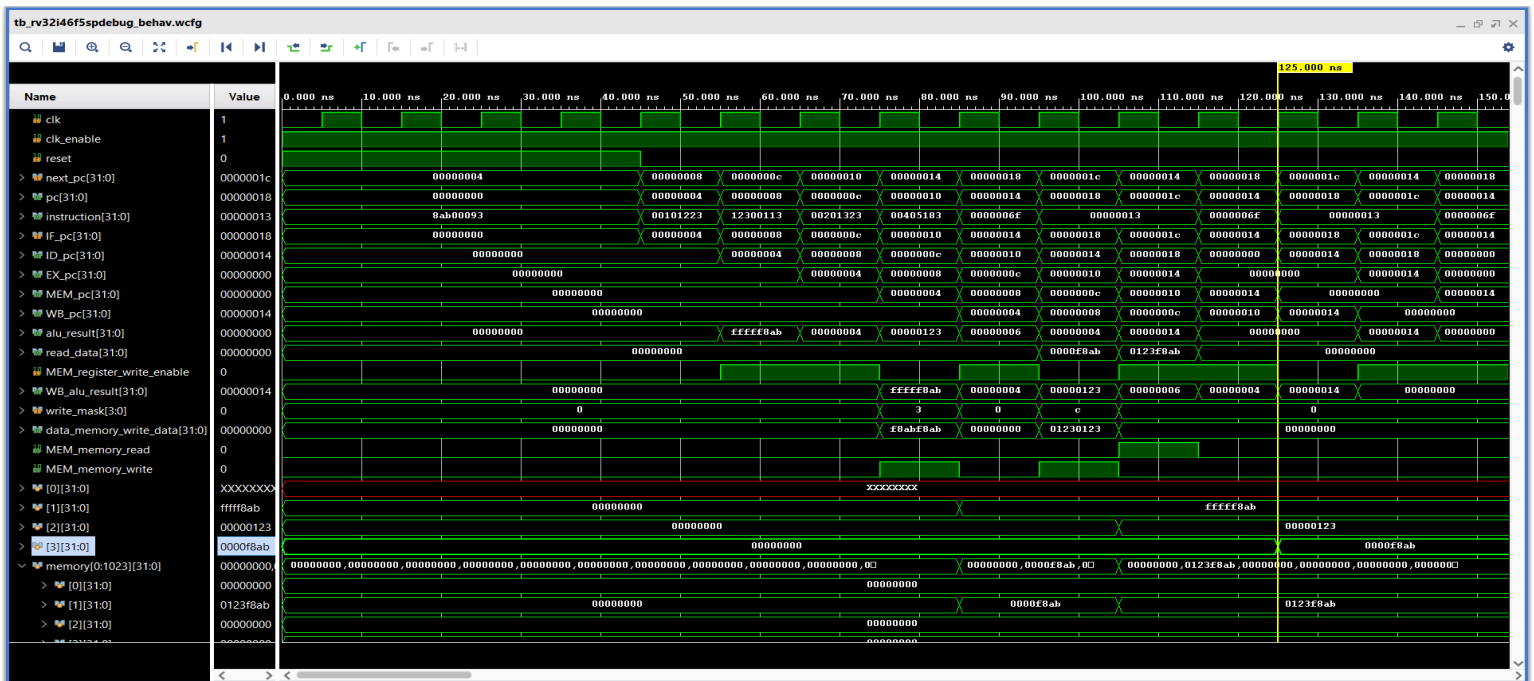
Instruction: lhu (load halfword unsigned)

Type: I-type

Instruction memory:

```
data[0] = {12'h8AB, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};
data[1] = {7'd0, 5'd1, 5'd0, `STORE_SH, 5'd4, `OPCODE_STORE};
data[2] = {12'h123, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};
data[3] = {7'd0, 5'd2, 5'd0, `STORE_SH, 5'd6, `OPCODE_STORE};
data[4] = {12'd4, 5'd0, `LOAD_LHU, 5'd3, `OPCODE_LOAD}; // lhu x3, 4(x0)
data[5] = 32'h0000006f; // jal x0,0
```

Simulation Result:



Comments:

The lhu instruction is fetched and advances smoothly through the decode and execute cycles to compute its target memory read address. During its fourth cycle in the MEM stage, the memory module outputs the complete stored word 0x0123f8ab, allowing the processor to precisely extract the lowest halfword 0xf8ab. In its fifth and final cycle, the operation successfully concludes by correctly zero-extending this extracted halfword to 0x0000f8ab just before updating the destination register in the WB stage.

8.

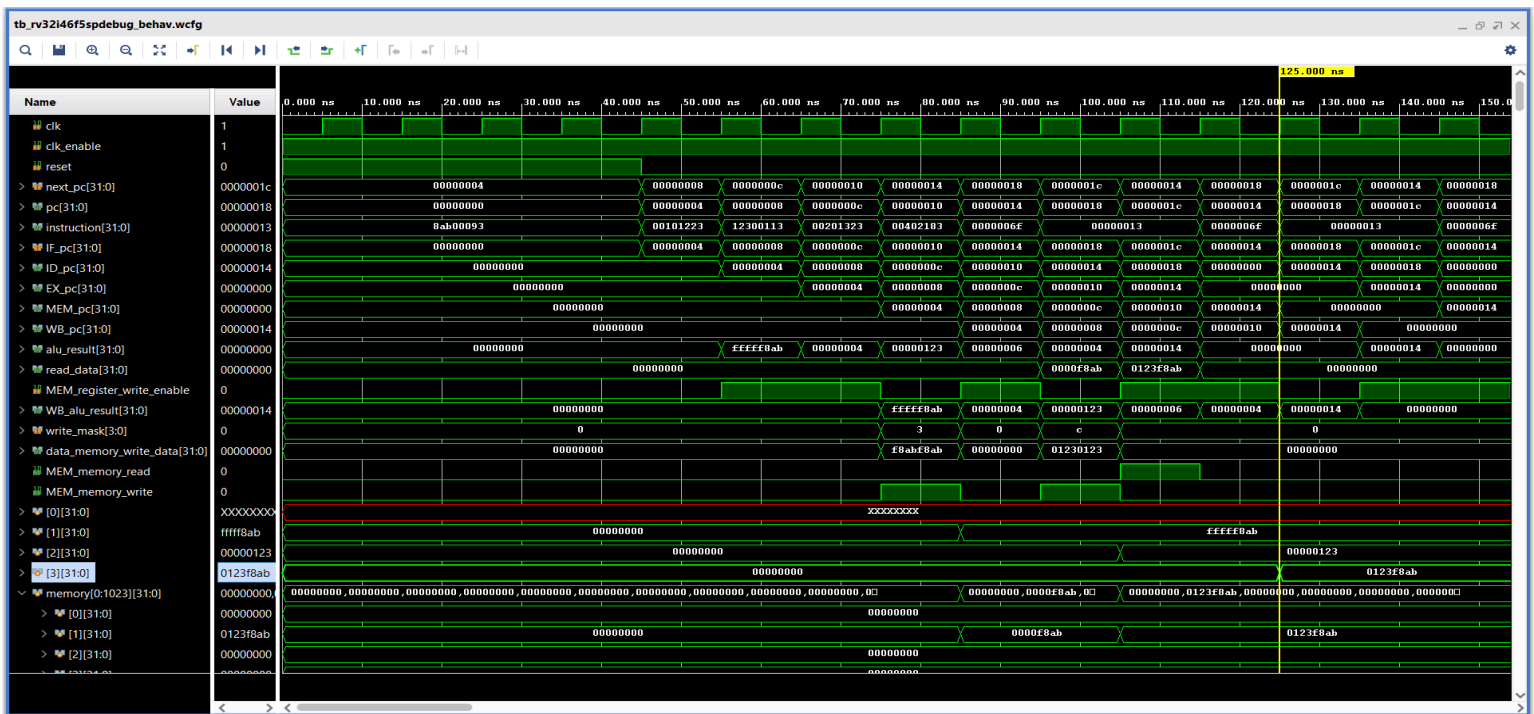
Instruction: lw (load word)

Type: I-type

Instruction memory:

```
data[0] = {12'h8AB, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};  
data[1] = {7'd0, 5'd1, 5'd0, `STORE_SH, 5'd4, `OPCODE_STORE};  
data[2] = {12'h123, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};  
data[3] = {7'd0, 5'd2, 5'd0, `STORE_SH, 5'd6, `OPCODE_STORE};  
data[4] = {12'd4, 5'd0, `LOAD_LW, 5'd3, `OPCODE_LOAD}; // lw x3, 4(x0)  
data[5] = 32'h0000006f; // jal x0,0
```

Simulation Result:



Comments:

The lw instruction is fetched and advances smoothly through the decode and execute cycles to compute its target memory read address. During its fourth cycle in the MEM stage, the memory module outputs the complete stored word 0x0123f8ab, allowing the processor to pass the full 32-bit value without any masking or extension. In its fifth and final cycle, the operation successfully concludes by writing this exact, unmodified word 0x0123f8ab directly into the destination register just before retiring in the WB stage.

9.

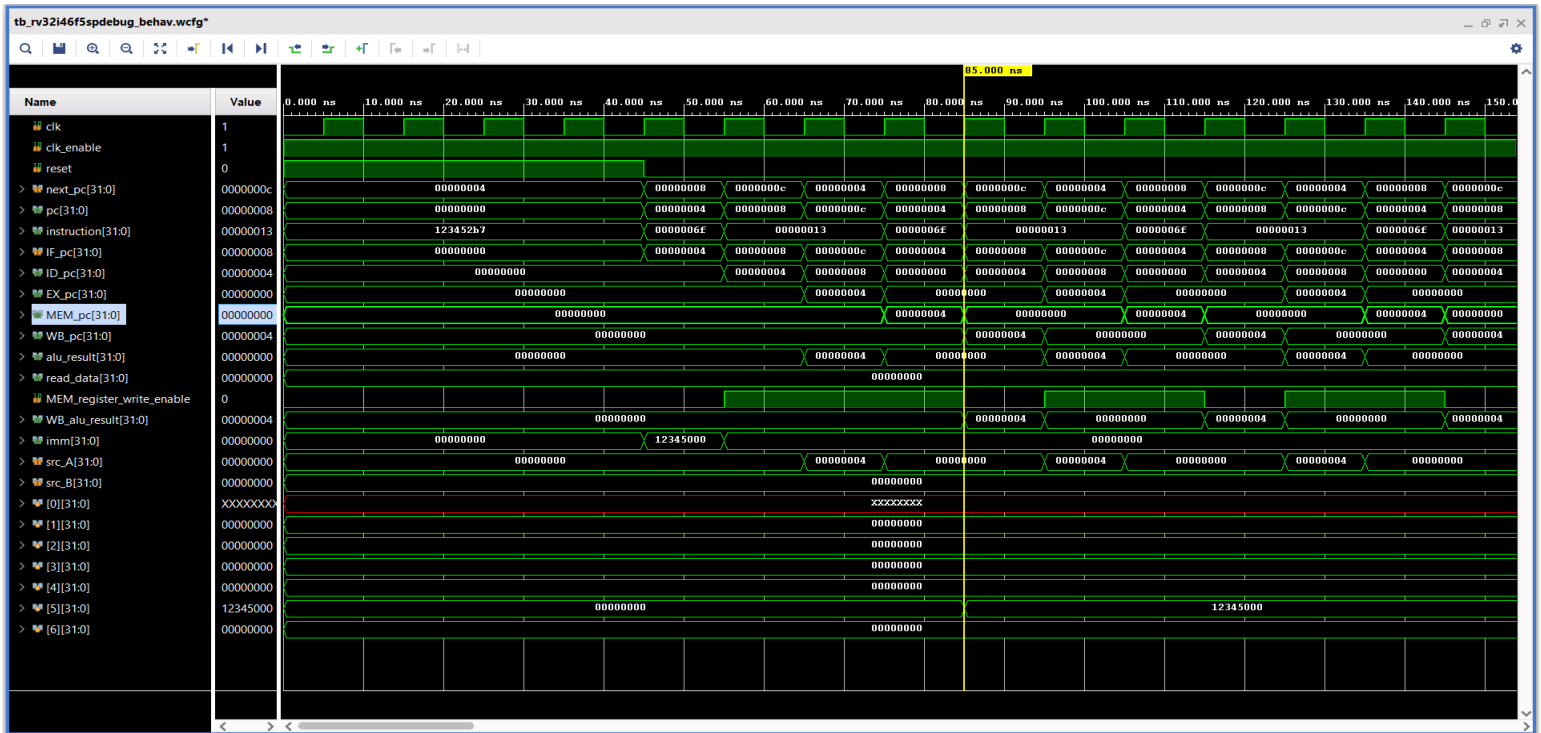
Instruction: lui (load upper immediate)

Type: U-type

Instruction memory:

```
data[0] = 32'h123452B7; // lui x5, 0x12345
data[1] = 32'h0000006f; // jal x0, 0 (Infinite loop to stop execution)
```

Simulation Result:



Comments:

The lui instruction is fetched at PC 0x00000004 and advances through the decode stage, successfully extracting the shifted upper immediate 0x12345000. During its third cycle in the EX stage, the ALU multiplexers correctly route this immediate alongside a zeroed input to compute the final payload 0x12345000. In its fifth and final cycle, the operation successfully concludes by writing this exact value into destination register x5 right before retiring in the WB stage.

10.

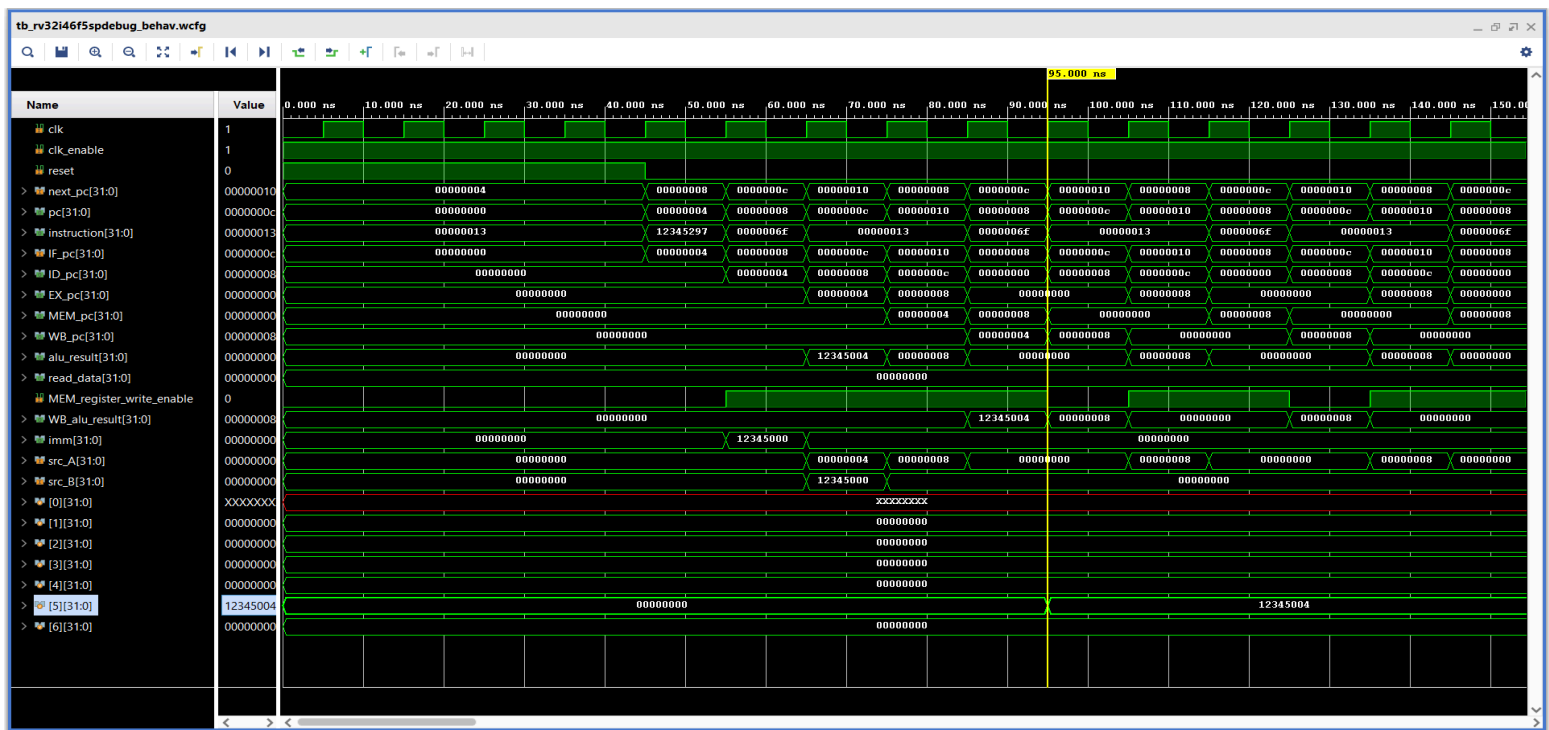
Instruction: auipc (add upper immediate to pc)

Type: U-type

Instruction memory:

```
data[0] = 32'h00000013; // nop (addi x0, x0, 0) - Placed at PC = 0
data[1] = 32'h12345297; // auipc x5, 0x12345 - Placed at PC = 4
data[2] = 32'h0000006f; // jal x0, 0
```

Simulation Result:



Comments:

The auipc instruction is fetched at PC 0x00000004 and advances through the decode stage, successfully extracting the shifted upper immediate 0x12345000. During its third cycle in the EX stage, the ALU multiplexers correctly route the instruction's own PC and this immediate to compute the target sum 0x12345004. In its fifth and final cycle, the operation concludes by successfully writing this computed address 0x12345004 into the destination register x5 right before retiring in the WB stage.

11.

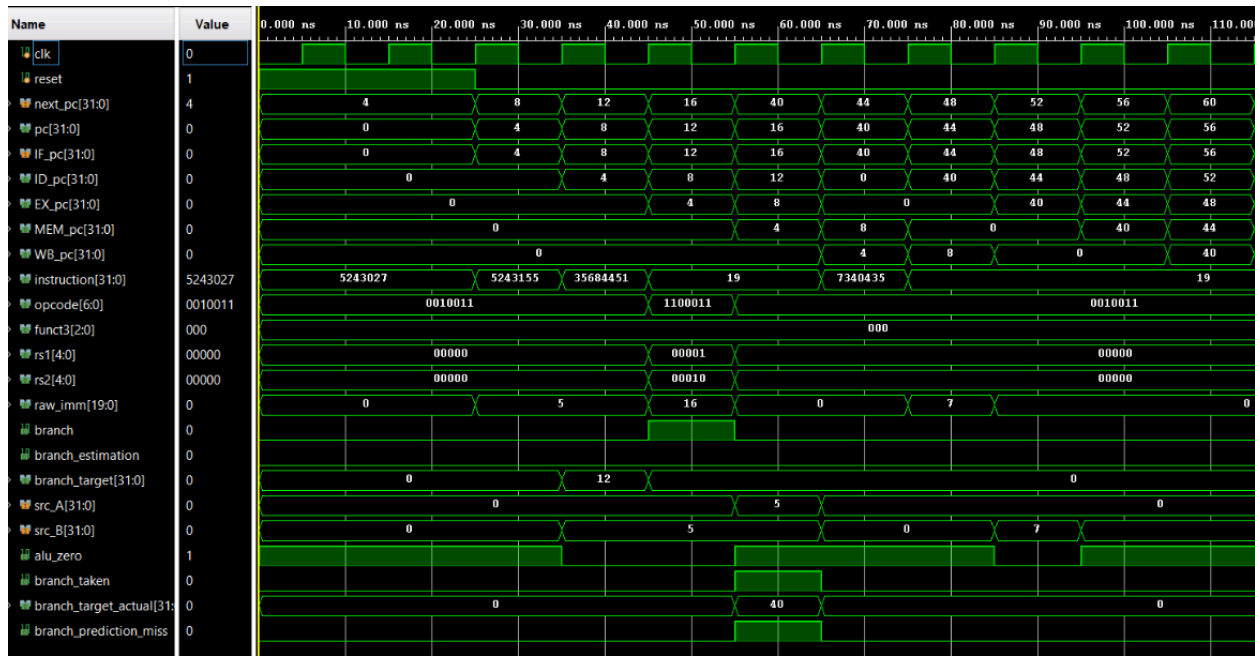
Instruction: beq (branch if equal)

Type: B-type

Instruction memory:

```
data[0] = {12'd5, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};  
data[1] = {12'd5, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};  
data[2] = {1'b0, 6'd1, 5'd2, 5'd1, `BRANCH_BEQ, 4'b0000, 1'b0, `OPCODE_BRANCH};  
data[10] = {12'd7, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};
```

Simulation Result:



Comments:

Registers x1 and x2 are both loaded with value 5. The beq instruction compares the values of these two registers using ALU by subtracting them, and since they are equal, the output of ALU is 0, and thus the alu_flag (the zero flag of ALU) is 1. The Branch Logic takes the alu_flag signal as input and produces a branch_taken = alu_zero = 1. Thus, the program counter value is added to the immediate value mentioned as per the instruction, that is, address number 8 + 32 = 40. After the execute cycle, flush occurs because of which, ID_pc and EX_pc are zero.

12.

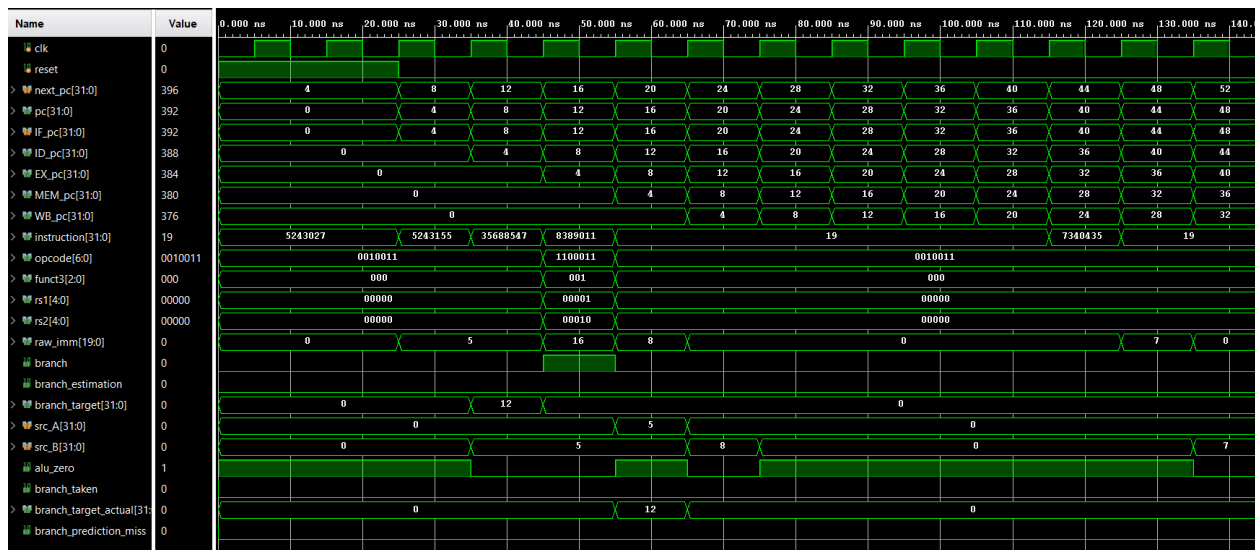
Instruction: bne (branch if not equal)

Type: B-type

Instruction memory:

```
data[0] = {12'd5, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};  
data[1] = {12'd5, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};  
data[2] = {1'b0, 6'd1, 5'd2, 5'd1, `BRANCH_BNE, 4'b0000, 1'b0, `OPCODE_BRANCH};  
data[3] = {12'd8, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};  
data[10] = {12'd7, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};
```

Simulation Result:



Comments:

The same registers, with the same values, are used for the bne instruction. It also compares the values by subtracting them using the ALU, and because they are equal, the alu_zero flag is 1. But a branch is supposed to happen when they are not equal, thus, the branch_taken is declared as $\sim \text{alu_zero}$. The former signal is thus 0, and no branch occurs; the new value of PC is PC + 4, that is, 12. It continues to fetch, decode, and execute the next instruction sequentially. The instruction that it was supposed to branch to is fetched only when PC reaches at its address in the instruction memory sequentially.

13.

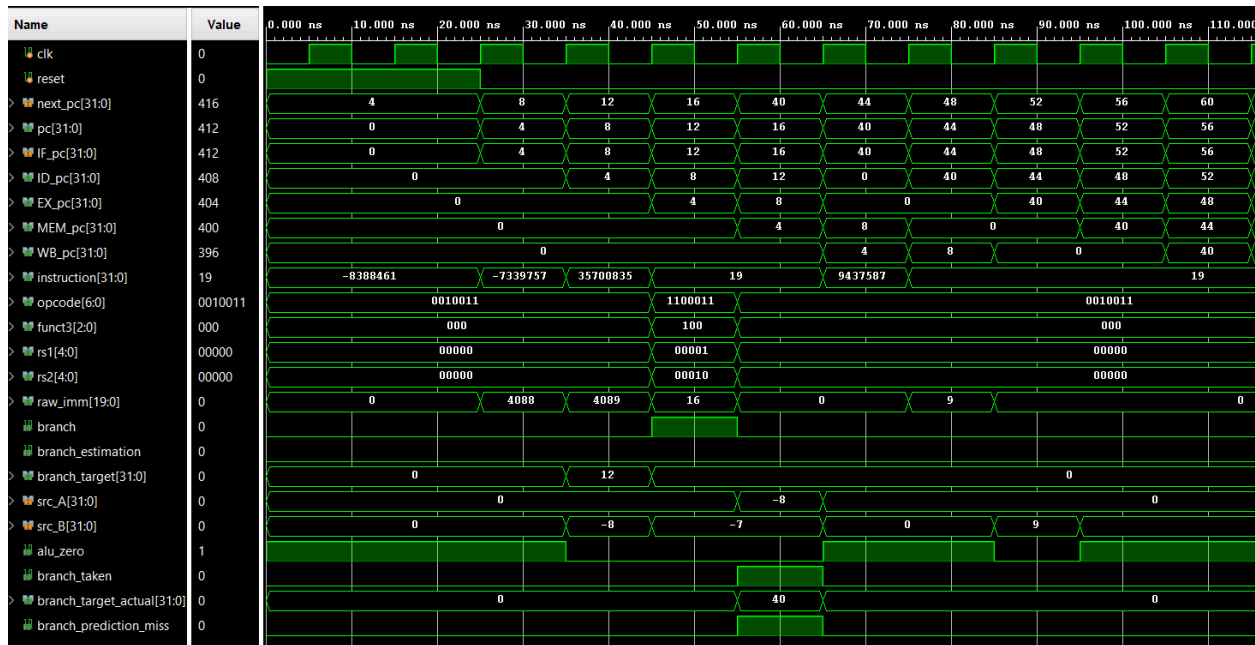
Instruction: blt (branch if less than)

Type: B-type

Instruction memory:

```
data[0] = {-12'd8, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};  
data[1] = {-12'd7, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};  
data[2] = {1'b0, 6'd1, 5'd2, 5'd1, `BRANCH_BLT, 4'b0000, 1'b0, `OPCODE_BRANCH};  
data[10] = {12'd9, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};
```

Simulation Result:



Comments:

Since signed comparison is allowed here, x1 is -8, and x2 is -7. The blt instruction effectively uses slt instruction between source operands A and B of the ALU, A being x1 and B being x2. Since $x1 < x2$, the slt sets the ALU output to 1. Consequently, the alu_zero flag is 0, but since this is less than, branch_taken is declared as $\sim \text{alu_zero}$, and thus, branch is taken, and PC goes to 40, as per the immediate value mentioned in the instruction.

14.

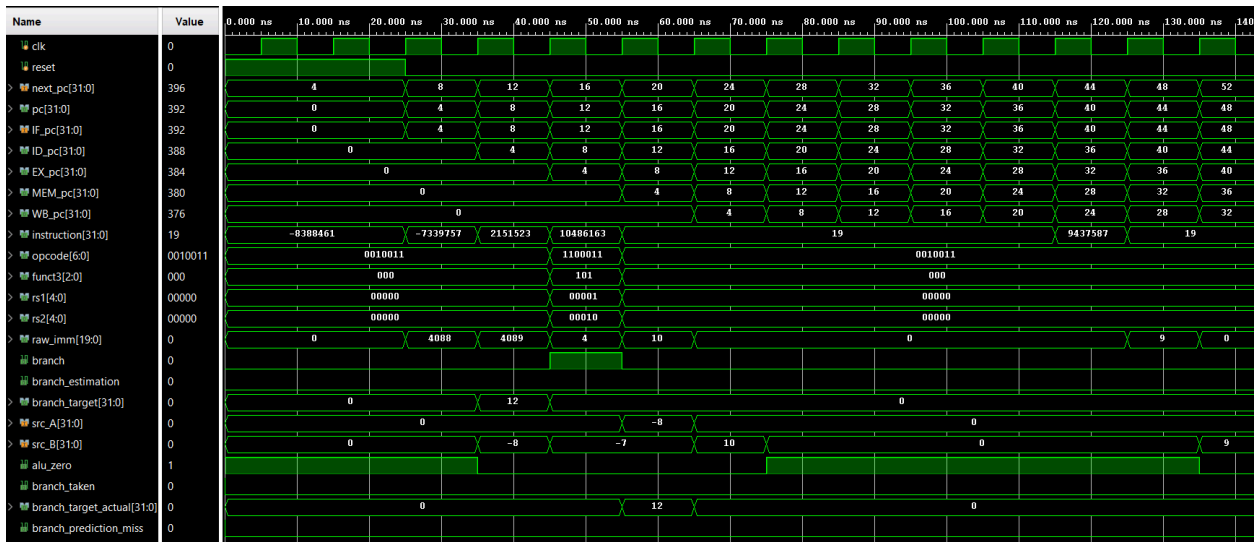
Instruction: bge (branch if greater than or equal to)

Type: B-type

Instruction memory:

```
data[0] = {-12'd8, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};
data[1] = {-12'd7, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};
data[2] = {1'b0, 6'd0, 5'd2, 5'd1, `BRANCH_BGE, 4'b0100, 1'b0, `OPCODE_BRANCH};
data[3] = {12'd10, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};
data[10] = {12'd9, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};
```

Simulation Result:



Comments:

With the same values in the registers x1 and x2 as above, bge is used. It also effectively uses slt if $\text{src_A} < \text{src_B}$, that is, $x1 < x2$. Since it is true in this case, ALU output is set to 1, and the alu_zero is 0. Since it was supposed to branch only with $x1 \geq x2$, branch_taken is declared as alu_zero, that is, 0 in this case. Thus, no branching is involved, and all the next instructions are executed before PC reaches the instruction that it would have pointed to had the branching occurred.

15.

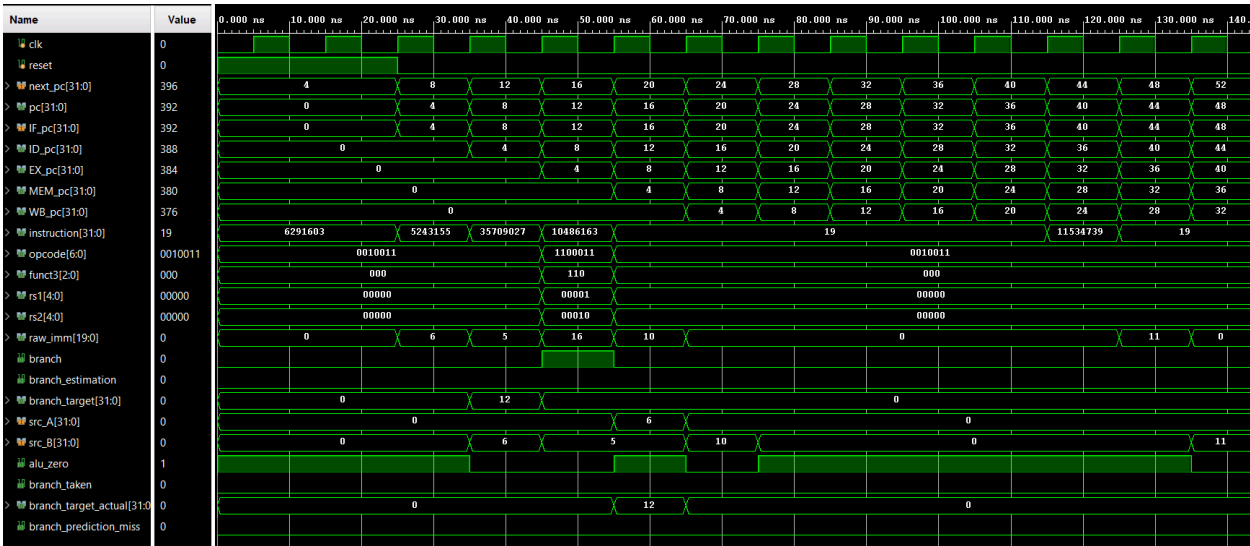
Instruction: bltu (branch if less than unsigned)

Type: B-type

Instruction memory:

```
data[0] = {12'd6, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};
data[1] = {12'd5, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};
data[2] = {1'b0, 6'd1, 5'd2, 5'd1, `BRANCH_BLTU, 4'b0000, 1'b0, `OPCODE_BRANCH};
data[3] = {12'd10, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};
data[10] = {12'd11, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};
```

Simulation Result:



Comments:

Similar to blt, with unsigned values. Branch not taken because $x1 > x2$.

16.

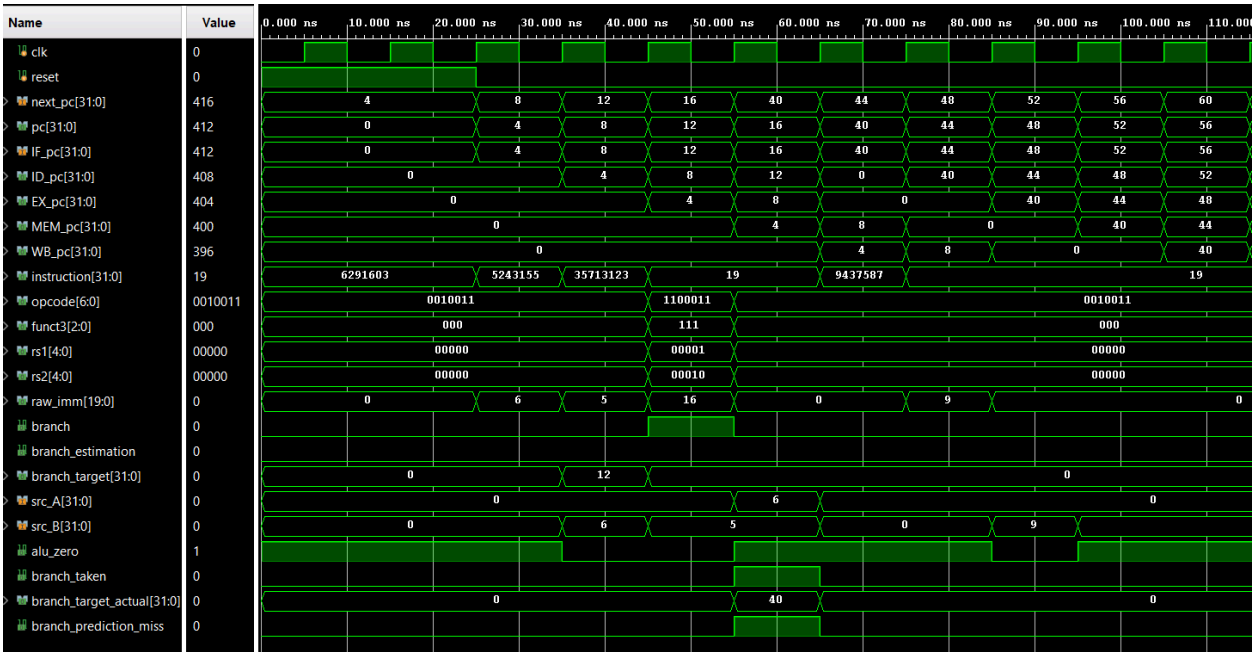
Instruction: bgeu (branch if greater than or equal to unsigned)

Type: B-type

Instruction memory:

```
data[0] = {12'd6, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};
data[1] = {12'd5, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};
data[2] = {1'b0, 6'd1, 5'd2, 5'd1, `BRANCH_BGEU, 4'b0000, 1'b0, `OPCODE_BRANCH};
data[10] = {12'd9, 5'd0, `ITYPE_ADDI, 5'd3, `OPCODE_ITYPE};
```

Simulation Result:



Comments:

Similar to bge, with unsigned values. Branch is taken because $x1 > x2$.

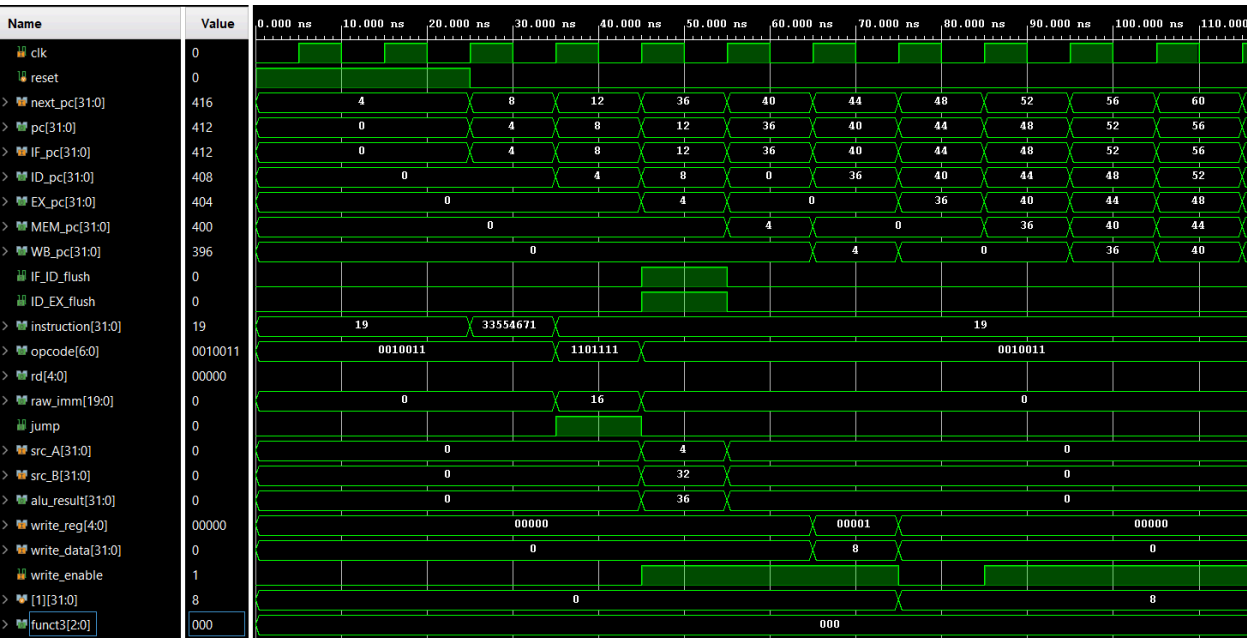
17.

Instruction: jal (jump and link)
Type: J-type

Instruction memory:

```
data[1] = {1'b0, 10'b0000010000, 1'b0, 8'b0, 5'd1, `OPCODE_JAL};
```

Simulation Result:



Comments: The simulation waveform successfully demonstrates the correct execution of a JAL (Jump and Link) instruction. Upon decoding the JAL opcode (1101111), the control unit accurately asserts the `jump` signal, which updates `next_pc` to the calculated target address and redirects the control flow. To handle the resulting control hazard, the pipeline appropriately asserts both `IF_ID_flush` and `ID_EX_flush`, clearing the sequentially fetched instructions to prevent incorrect execution. Finally, the linking mechanism is verified during the writeback stage, where `write_enable` goes high to store the return address (PC + 4, visible on `write_data`) into the destination register (`write_reg` 00001), confirming the complete and correct operation of the instruction.

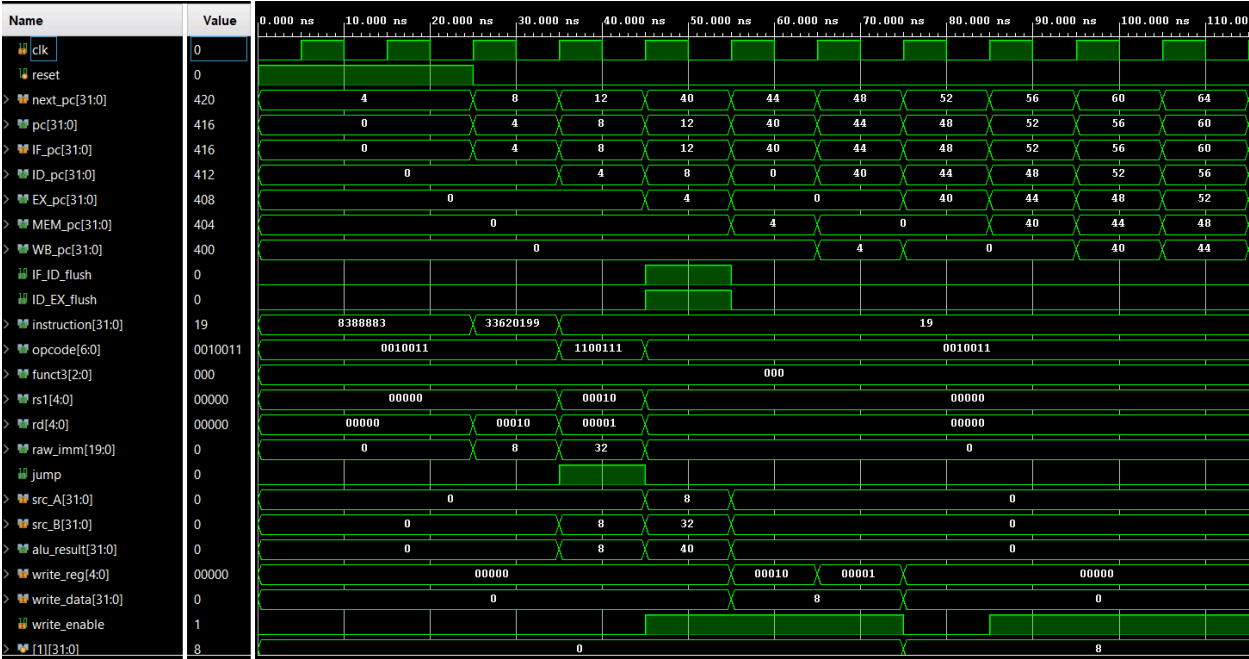
18.

Instruction: jalr (jump and link register)
Type: I-type

Instruction memory:

```
data[0] = {12'd8, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};  
data[1] = {12'd32, 5'd2, 3'b000, 5'd1, `OPCODE_JALR};
```

Simulation Result:



Comments:

The simulation waveform captures the correct execution of a JALR instruction, identified by the opcode 1100111. During the execute stage, the ALU calculates the jump target by adding the base address from register rs1 (00010, which provides a value of 8 on src_A) and the immediate value (32 on src_B), resulting in a target address of 40. The control flow is successfully redirected as the jump signal is asserted and next_pc updates to this new target (40). To mitigate the resulting control hazard, the pipeline correctly asserts both IF_ID_flush and ID_EX_flush, purging the incorrectly fetched sequential instructions. Finally, the linking behavior is verified as write_enable goes high to store the return address (PC + 4, which is 12 since the JALR is at PC 8) into the destination register write_reg (00001, or x1/ra), confirming the complete operation of the jump-and-link sequence.

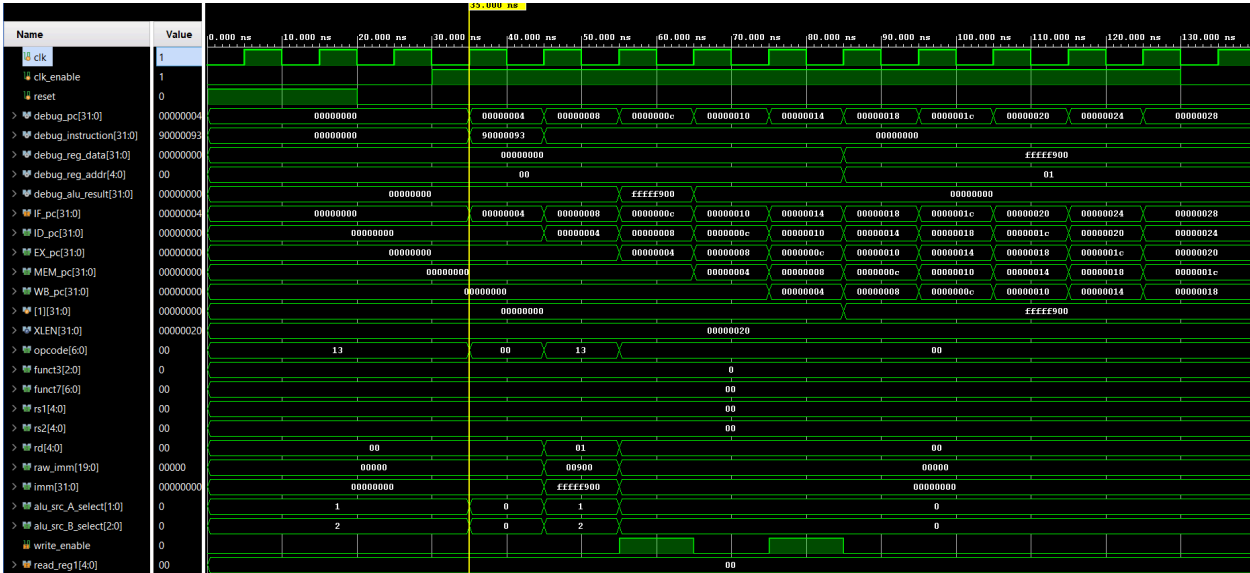
19.

Instruction: addi (add immediate)
Type: I-type

Instruction memory:

```
data[1] = {12'h900, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // addi x1,x0,0x900
```

Simulation Result:



Comments:

The addi instruction is fetched at PC 0x00000004 and advances into the decode stage, where the processor extracts the 12-bit immediate operand and identifies x0 as the source register. Since register x0 always contains the value 0, the ALU receives 0 along with the immediate operand during the execute stage. The ALU performs the addition operation and produces a result equal to the immediate value. During the write-back stage, this computed result is written into the destination register x1, completing the instruction.

20.

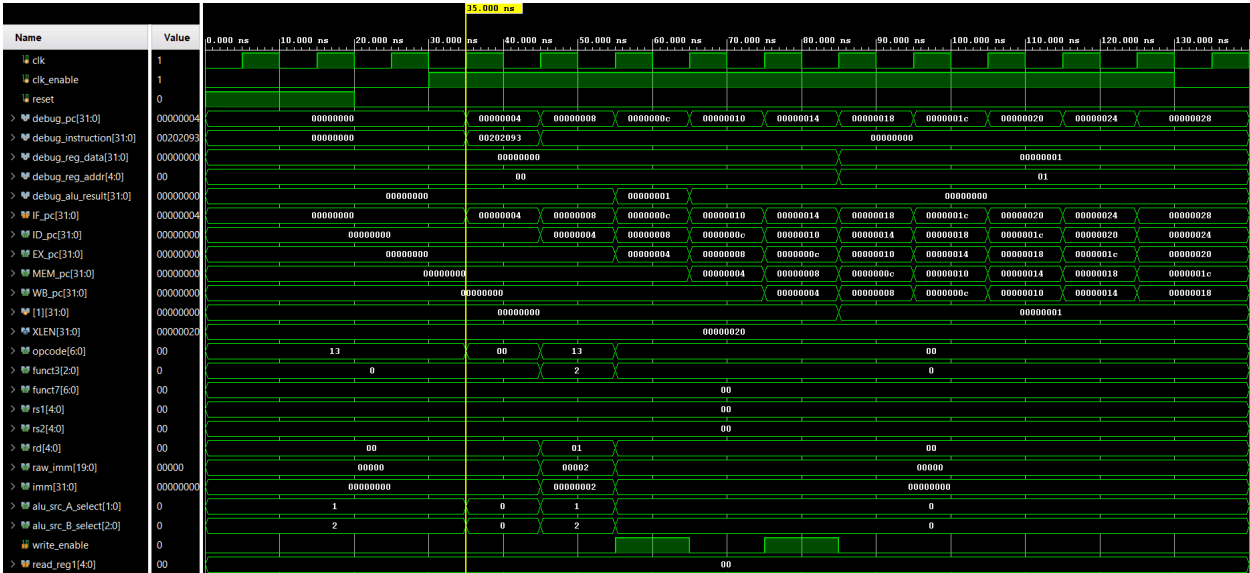
Instruction: slti (set less than)

Type: I-type

Instruction memory:

```
data[1] = {12'h002, 5'd0, `ITYPE_SLTI, 5'd1, `OPCODE_ITYPE}; // slti x1,x0,2
```

Simulation Result:



Comments:

The `slti` instruction is fetched at PC 0x00000004 and proceeds through the decode stage, where the immediate value is extracted and `x0` is identified as the source register. In the execute stage, the ALU performs a signed comparison between the value stored in `x0` and the immediate operand. If the value in `x0` is smaller than the immediate value, the ALU produces 1, otherwise it produces 0. This result is written into the destination register `x1` during the write-back stage, after which the instruction retires.

21.

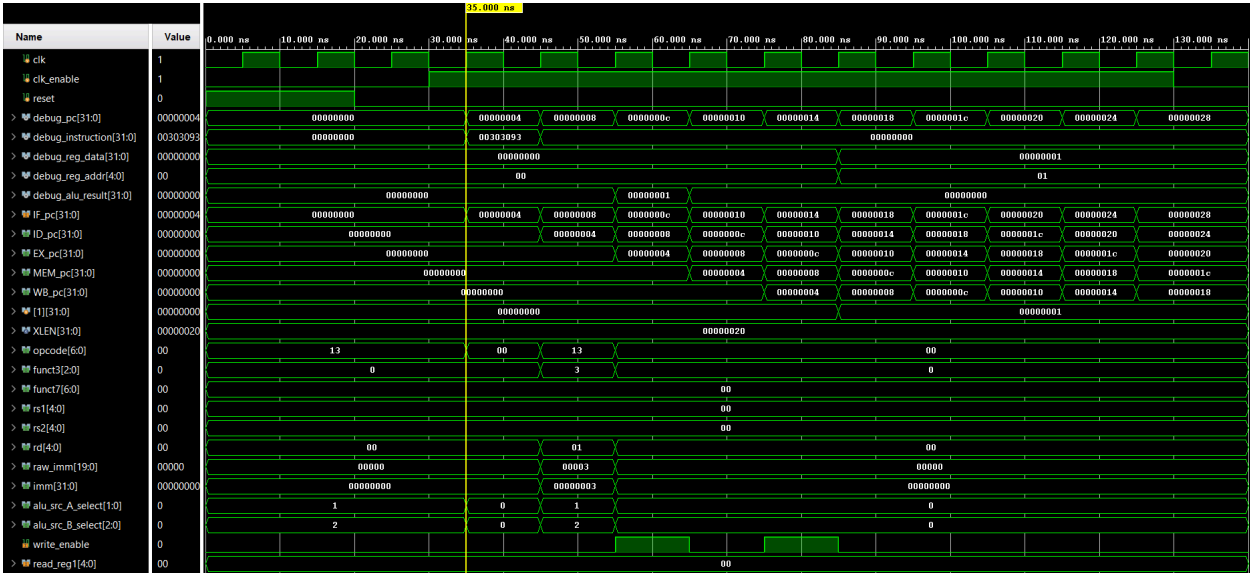
Instruction: sltiu (set less than unsigned)

Type: I-type

Instruction memory:

```
data[1] = {12'h003, 5'd0, `ITYPE_SLTIU, 5'd1, `OPCODE_ITYPE}; // sltiu x1,x0,3
```

Simulation Result:



Comments:

The `sltiu` instruction is fetched at PC 0x00000004 and advances into the decode stage, where the immediate operand and source register x0 are extracted. In the execute stage, the ALU performs an unsigned comparison between the value stored in x0 and the immediate value. The ALU generates 1 if the register value is smaller than the immediate value, otherwise 0. During the write-back stage, the resulting value is written into the destination register x1, completing the instruction execution.

22.

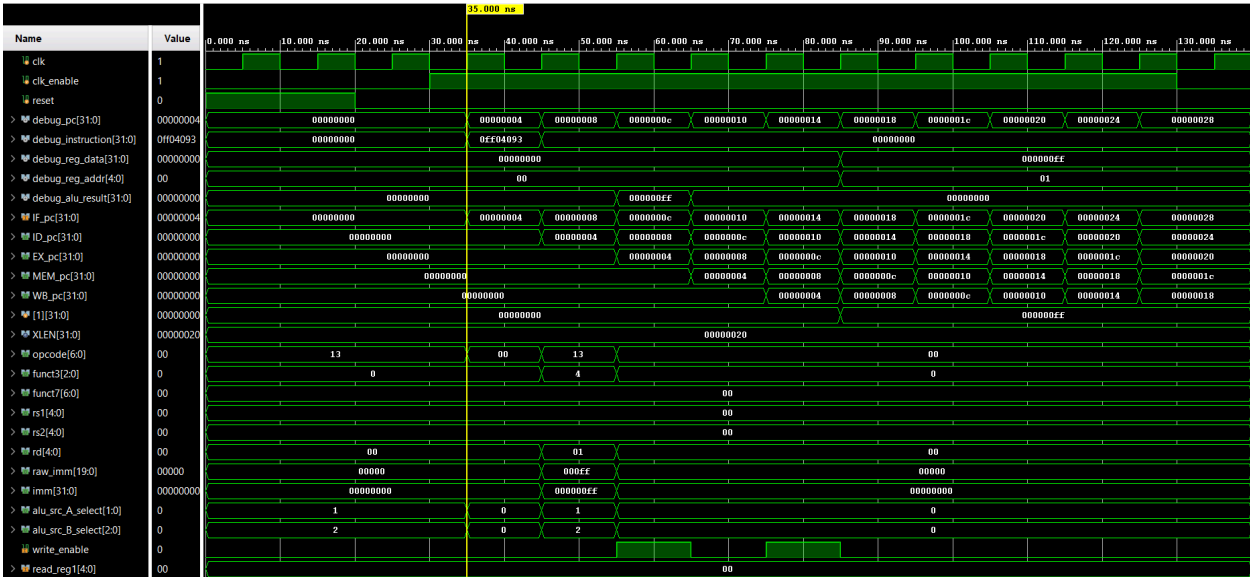
Instruction: xori (xor immediate)

Type: I-type

Instruction memory:

```
data[1] = {12'h0FF, 5'd0, `ITYPE_XORI, 5'd1, `OPCODE_ITYPE}; // xori x1,x0,0xFF
```

Simulation Result:



Comments:

The xori instruction is fetched at PC 0x00000004 and moves into the decode stage, where the processor extracts the immediate operand and identifies x0 as the source register. During the execute stage, the ALU performs a bitwise XOR operation between the value stored in x0 and the immediate value. The resulting bit pattern is generated by the ALU and forwarded to the write-back stage, where it is written into x1, completing the instruction.

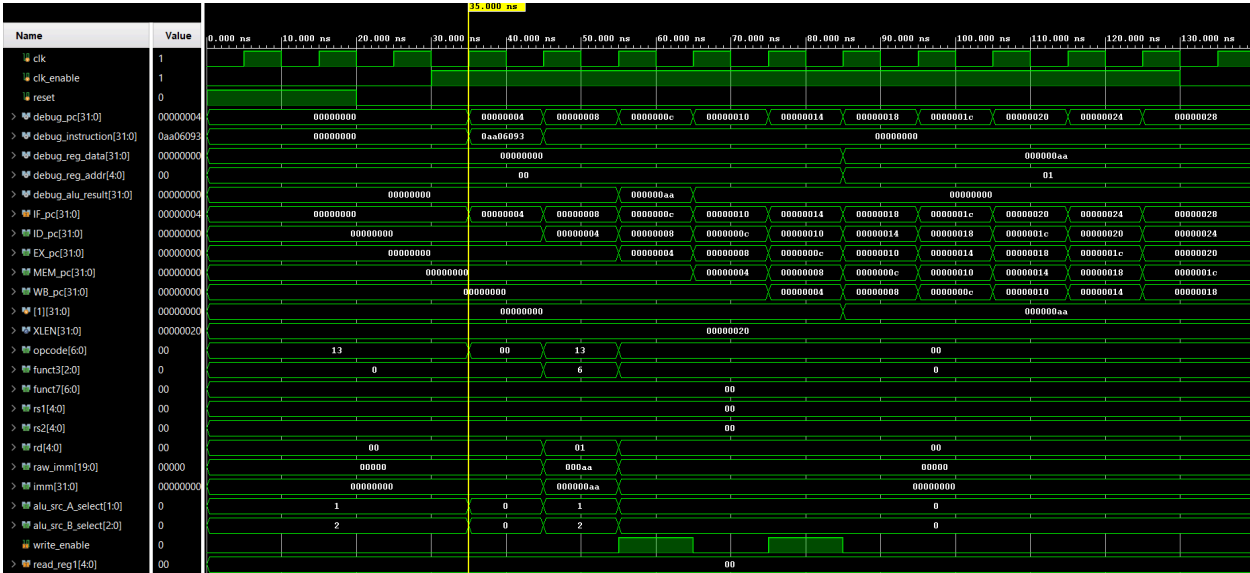
23.

Instruction: ori (or immediate)
Type: I-type

Instruction memory:

```
data[1] = {12'h0AA, 5'd0, `ITYPE_ORI, 5'd1, `OPCODE_ITYPE}; // ori x1,x0,0xAA
```

Simulation Result:



Comments:

The `ori` instruction is fetched at PC 0x00000004 and advances through the decode stage, where the processor extracts the immediate operand and identifies x0 as the source register. In the execute stage, the ALU performs a bitwise OR operation between the value stored in x0 and the immediate value. The resulting value is then written into the destination register x1 during the write-back stage, completing the instruction.

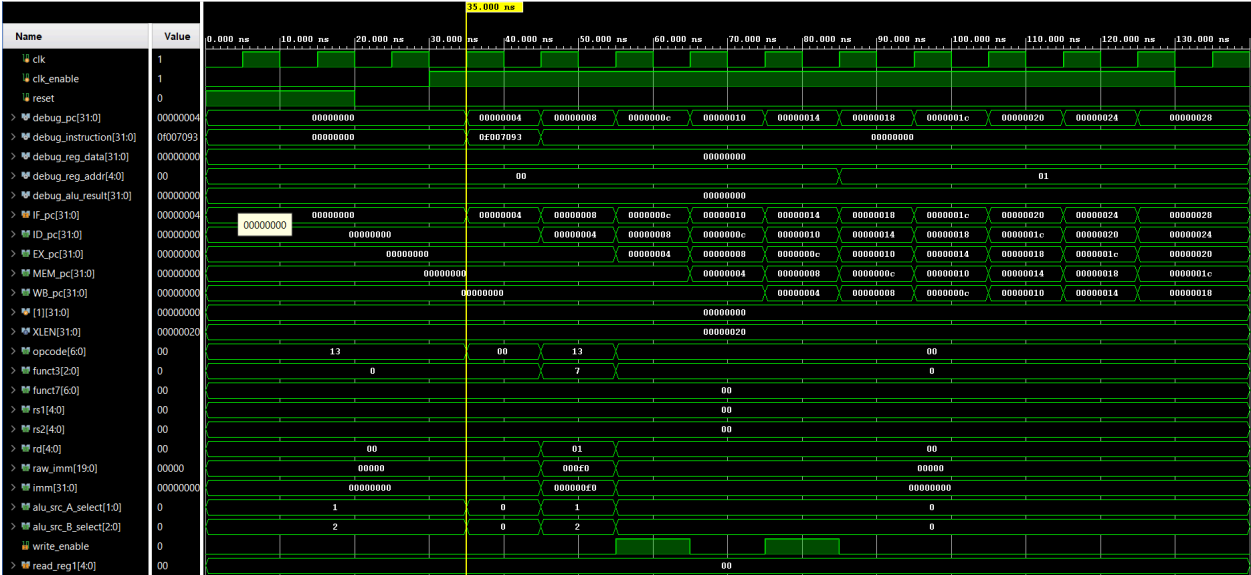
24.

Instruction: andi (and immediate)
Type: I-type

Instruction memory:

```
data[1] = {12'h0F0, 5'd0, `ITYPE_ANDI, 5'd1, `OPCODE_ITYPE}; // andi x1,x0,0xF0
```

Simulation Result:



Comments:

The `andi` instruction is fetched at PC `0x00000004` and proceeds through the decode stage, where the processor extracts the immediate operand and identifies `x0` as the source register. During the execute stage, the ALU performs a bitwise AND operation between the value stored in `x0` and the immediate value. The ALU generates the resulting value and forwards it to the write-back stage, where it is written into `x1`, after which the instruction retires.

25.

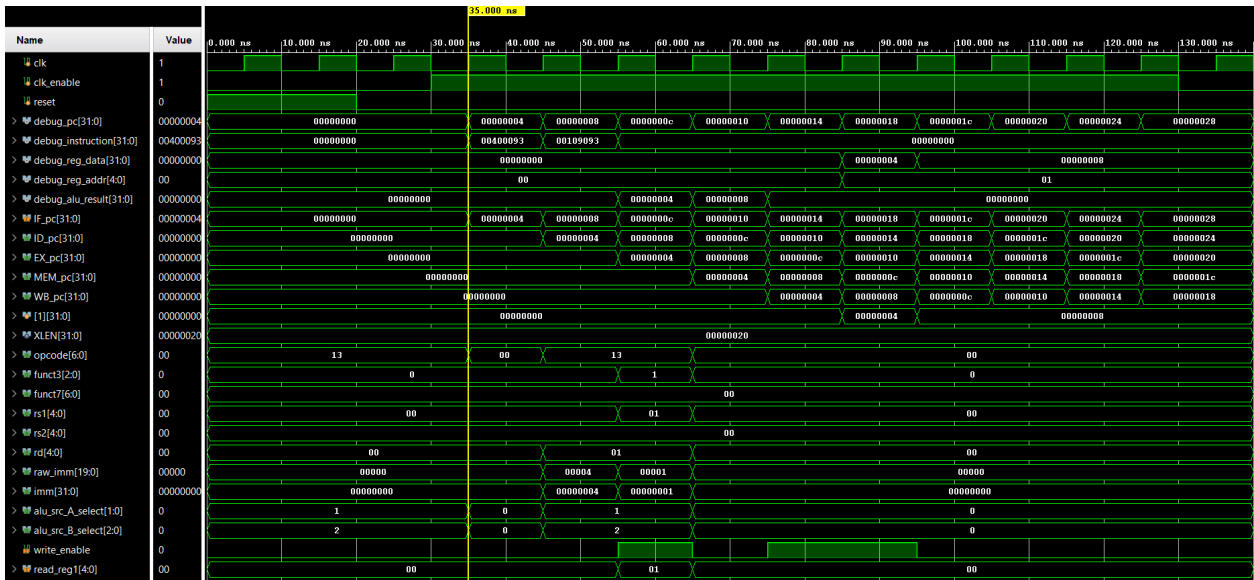
Instruction: slli (shift left logical)

Type: I-type

Instruction memory:

```
data[1] = {12'h004, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 4
data[2] = {7'b0000000,5'd1,5'd1,`ITYPE_SLLI,5'd1,`OPCODE_ITYPE}; // slli x1,x1,1
```

Simulation Result:



Comments:

The slli instruction is fetched at PC 0x00000008 and advances through the decode stage, where the processor extracts the shift amount 1 and identifies x1 as the source register. During the execute stage, the ALU performs a logical left shift on the value stored in x1. Since x1 contains the value 4, the shift operation produces the result 8. In the final write-back stage, this computed value is written into destination register x1, completing the instruction.

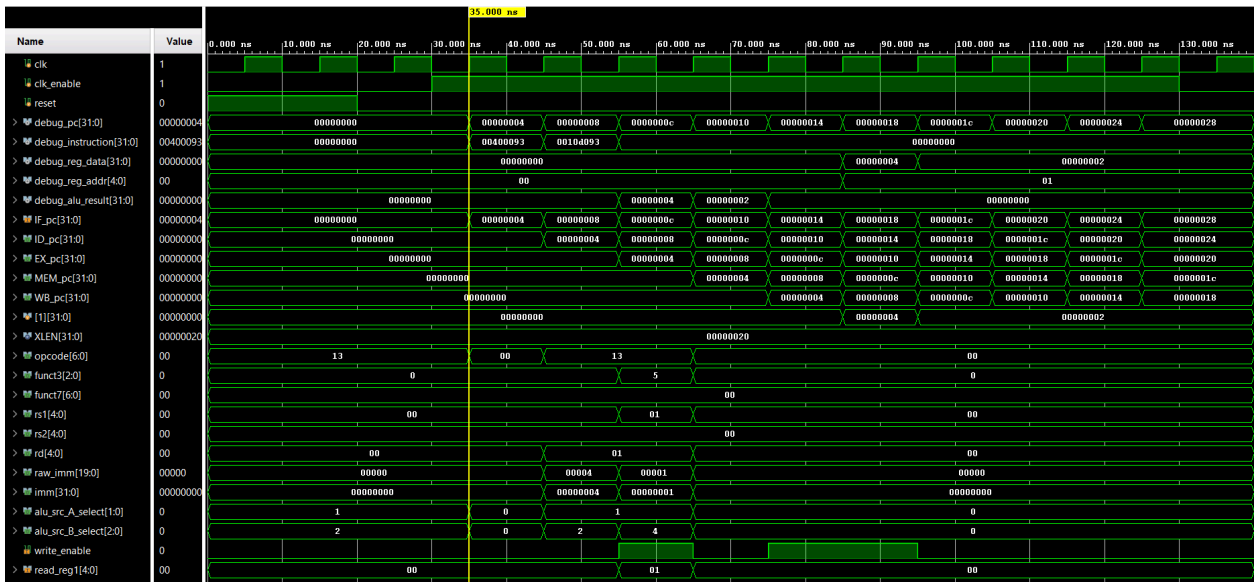
26.

Instruction: srli (shift right logical)
Type: I-type

Instruction memory:

```
data[1] = {12'h004, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 4
data[2] = {7'b0000000, 5'd1, 5'd1, `ITYPE_SRXI, 5'd1, `OPCODE_ITYPE}; // srli x1,x1,1
```

Simulation Result:



Comments:

The srli instruction is fetched at PC 0x00000008 and proceeds through the decode stage, where the processor extracts the shift amount 1 and identifies x1 as the source register. In the execute stage, the ALU performs a logical right shift on the value stored in x1, shifting the bits toward the least significant side while inserting zeros in the most significant positions. With x1 = 4, the operation produces the result 2, which is written back into x1 during the write-back stage before the instruction retires.

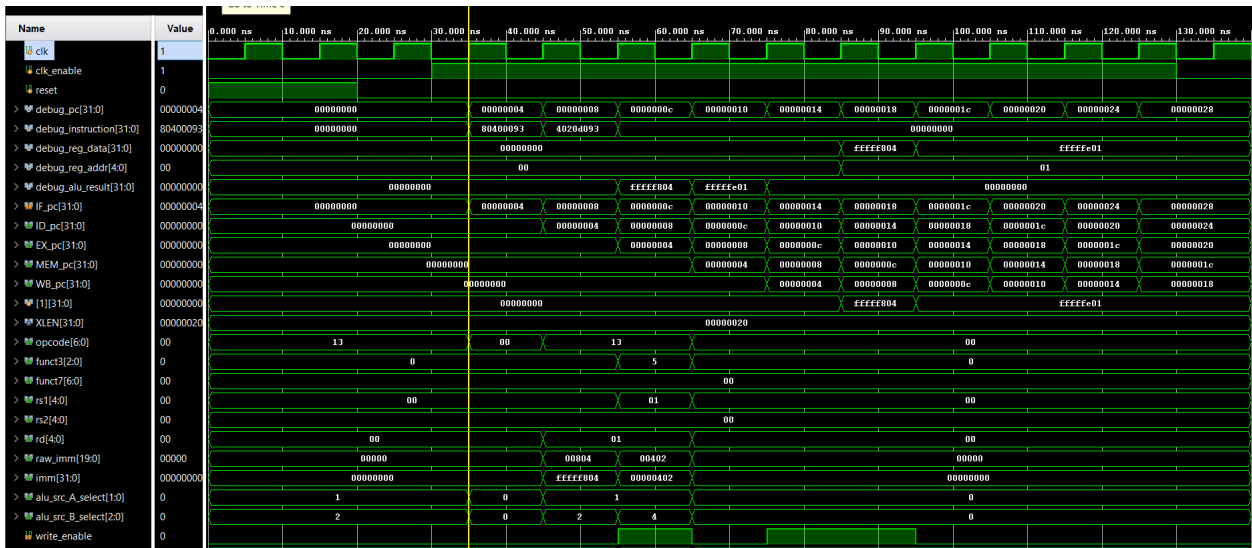
27.

Instruction: srai (shift right arithmetic)
Type: I-type

Instruction memory:

```
data[1] = {12'h804, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 4
data[2] = {7'b0100000, 5'd2, 5'd1, `ITYPE_SRXI, 5'd1, `OPCODE_ITYPE}; // srai x1,x1,1
```

Simulation Result:



Comments:

The srai instruction is fetched at PC 0x00000008 and advances through the decode stage, where the processor extracts the shift amount 2 and identifies x1 as the source register. During the execute stage, the ALU performs an arithmetic right shift, shifting the bits of x1 to the right while preserving the sign bit. Since x1 holds the value ffff804, the resulting value is ffff801, which is written into destination register x1 during the write-back stage, completing the instruction execution.

28.

Instruction: Sub

Type: R-type

Instruction memory:

```
data[0] = {12'd10, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 10
```

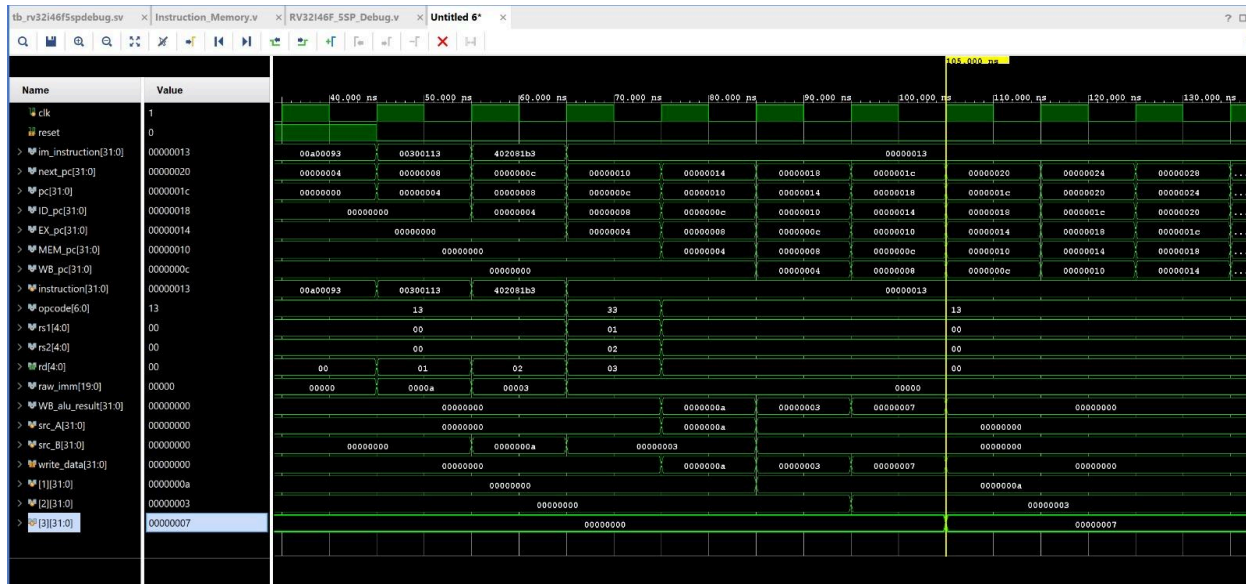
```
data[1] = {12'd3, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE}; // x2 = 3
```

```
// RAW hazard on x1 and x2
```

```
data[2] = {7'b0100000, 5'd2, 5'd1, `RTYPE_ADDSUB, 5'd3, `OPCODE_RTYPE};
```

```
// sub x3, x1, x2
```

Simulation Result:



Comments:

The first addi instruction is fetched at PC = 0x00000000, decoded to extract the immediate 10 and read x0, and the ALU writes the result into x1. The second addi at PC = 0x00000004 adds immediate 3 to x0 and writes the result into x2. The sub instruction at PC = 0x00000008 is decoded as an R-type instruction and reads x1 and x2, introducing a RAW dependency, after which the ALU performs the subtraction ($x1 - x2$) and writes the result into x3 before the instruction retires.

29.

Instruction: sll

Type: R-type

Instruction memory:

```
data[0] = {12'd4, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 4
```

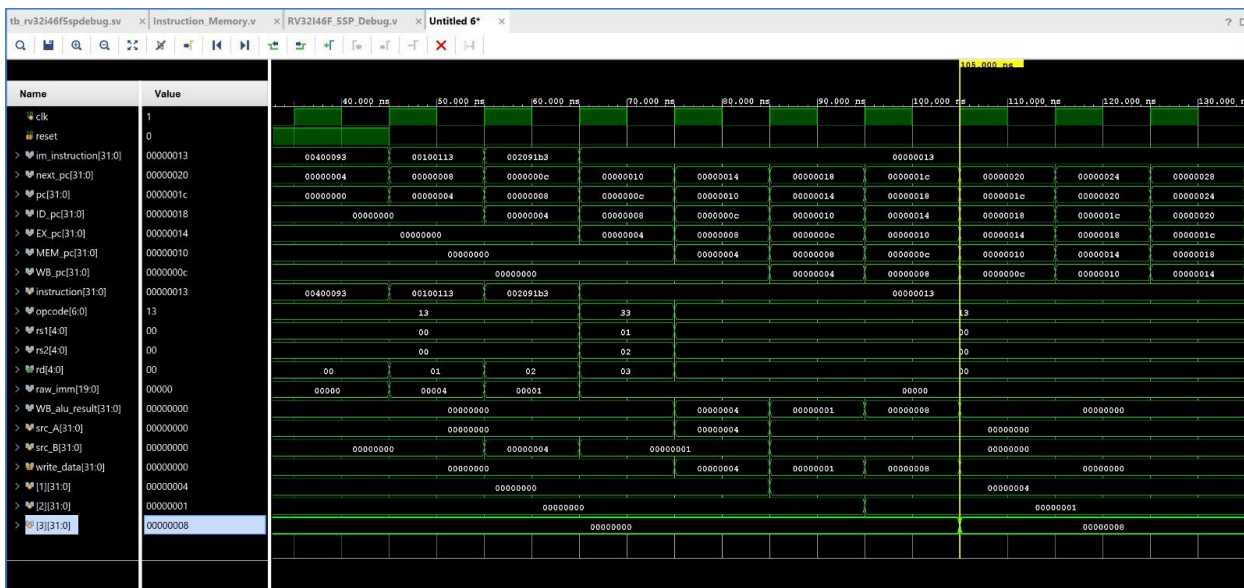
```
data[1] = {12'd1, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE}; // x2 = 1
```

```
// Uses both → RAW hazard
```

```
data[2] = {7'b0000000, 5'd2, 5'd1, 3'b001, 5'd3, `OPCODE_RTYPE};
```

```
// sll x3, x1, x2
```

Simulation Result:



Comments:

The sll instruction is fetched at PC = 0x00000008 and decoded as an R-type instruction, where the processor reads the source registers x1 and x2, introducing a RAW dependency. During the execute stage, the ALU performs a logical left shift of the value in x1 by the amount specified in x2. The result is then written back into the destination register x3, after which the instruction retires.

30.

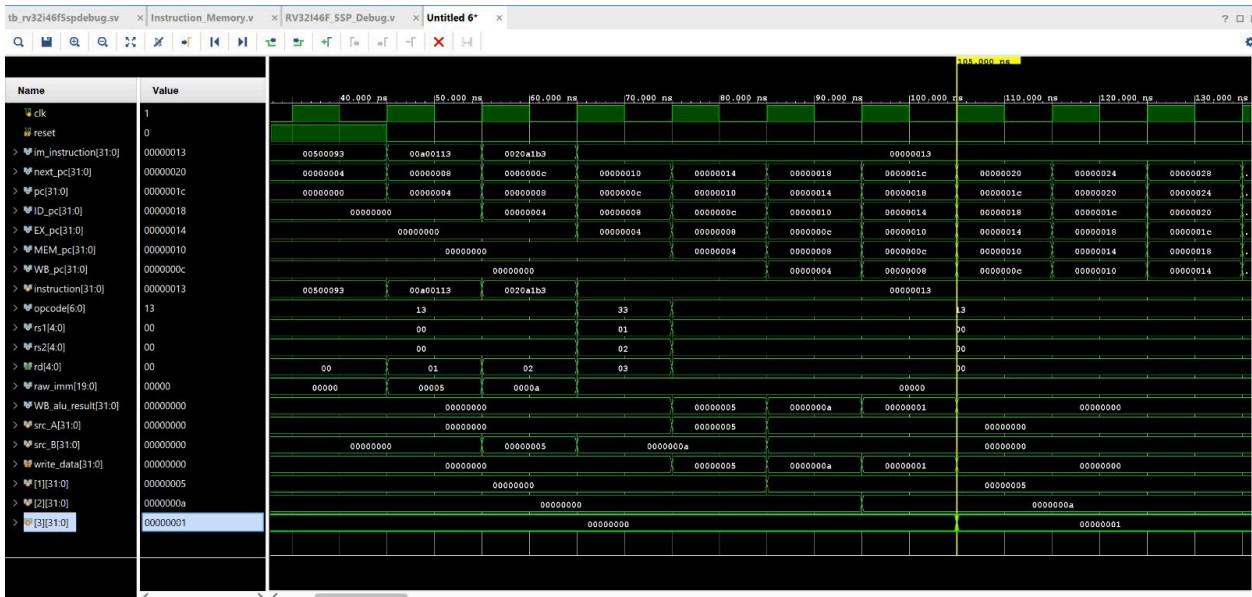
Instruction: slt
Type: R-type

Instruction memory:

```
// Initialize
data[0] = {12'd5, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 5
data[1] = {12'd10, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE}; // x2 = 10

// Compare
data[2] = {7'b0000000, 5'd2, 5'd1, 3'b010, 5'd3, `OPCODE_RTYPE};
// slt x3, x1, x2 → x3 = 1
```

Simulation Result:



Comments:

The slt instruction is fetched at PC = 0x00000008 and decoded as an R-type instruction, where the processor reads the source registers x1 and x2, introducing a RAW dependency. During the execute stage, the ALU compares the values in x1 and x2 and sets the result to 1 if x1 is less than x2, otherwise 0. The result is then written back into the destination register x3, after which the instruction retires.

31.

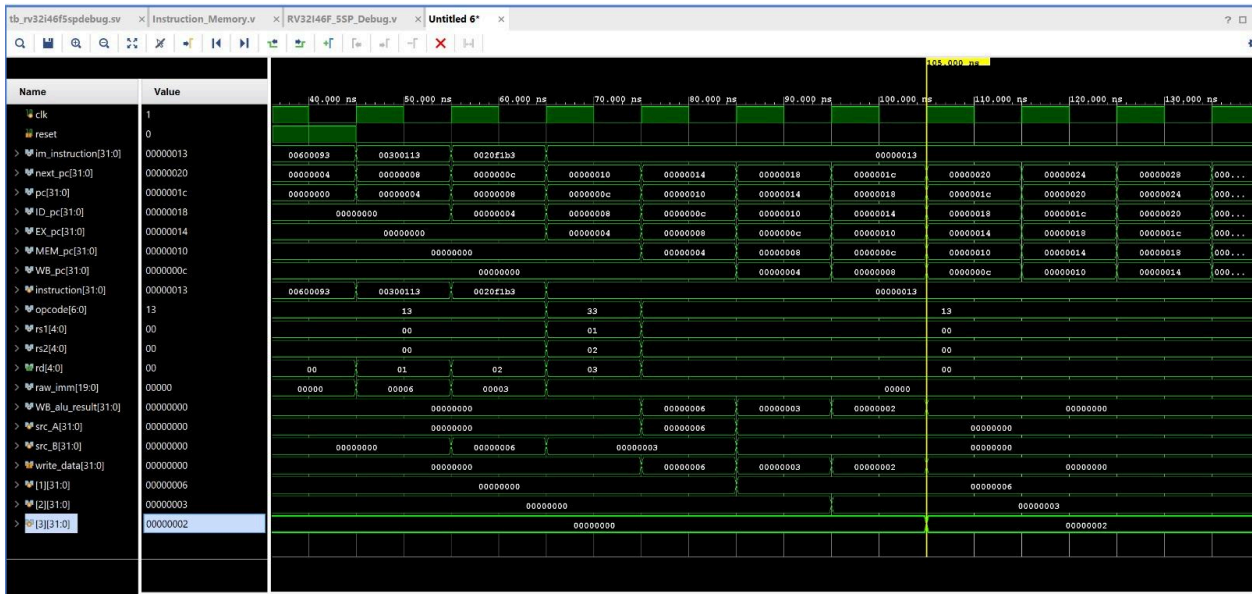
Instruction: and
Type: R-type

Instruction memory:

```
// Initialize
data[0] = {12'd6, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 6 (110)
data[1] = {12'd3, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE}; // x2 = 3 (011)

// AND
data[2] = {7'b00000000, 5'd2, 5'd1, 3'b111, 5'd3, `OPCODE_RTYPE};
// and x3, x1, x2 → x3 = 2 (010)
```

Simulation Result:



Comments:

The and instruction is fetched at PC = 0x00000008 and decoded as an R-type instruction, where the processor reads the source registers x1 and x2, introducing a RAW dependency. During the execute stage, the ALU performs a bitwise AND operation between the values in x1 and x2. The result is then written back into the destination register x3, after which the instruction retires.

32.

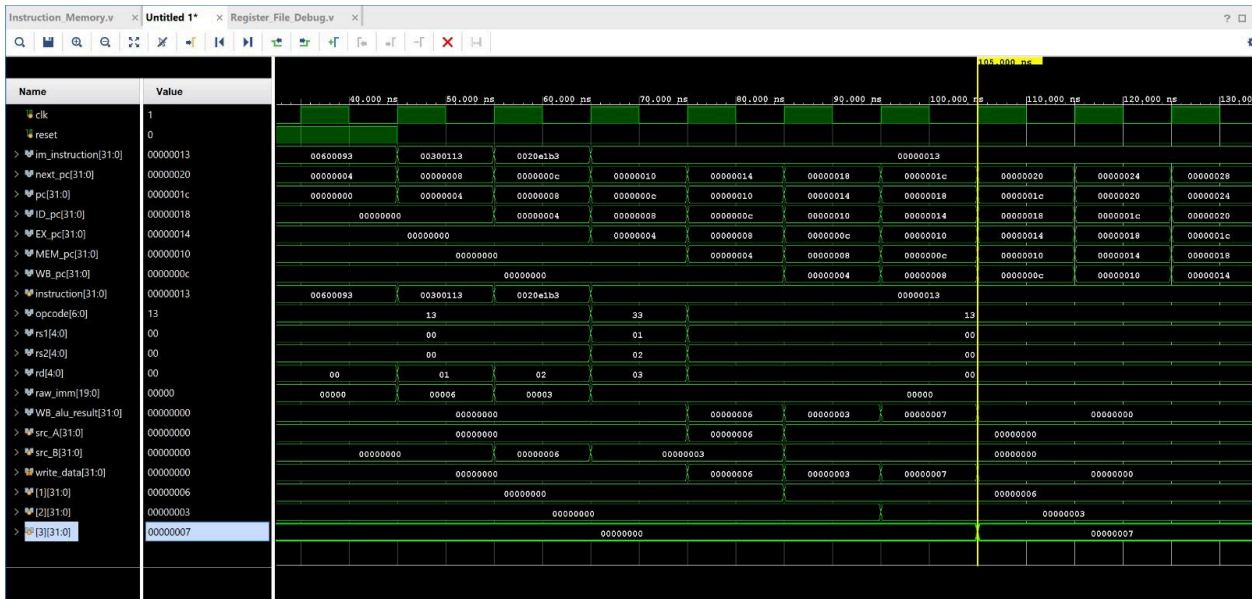
Instruction: or
Type: R-type

Instruction memory:

```
// Initialize
data[0] = {12'd6, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 6 (110)
data[1] = {12'd3, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE}; // x2 = 3 (011)

// OR
data[2] = {7'b0000000, 5'd2, 5'd1, 3'b110, 5'd3, `OPCODE_RTYPE};
// or x3, x1, x2 → x3 = 7 (111)
```

Simulation Result:



33.

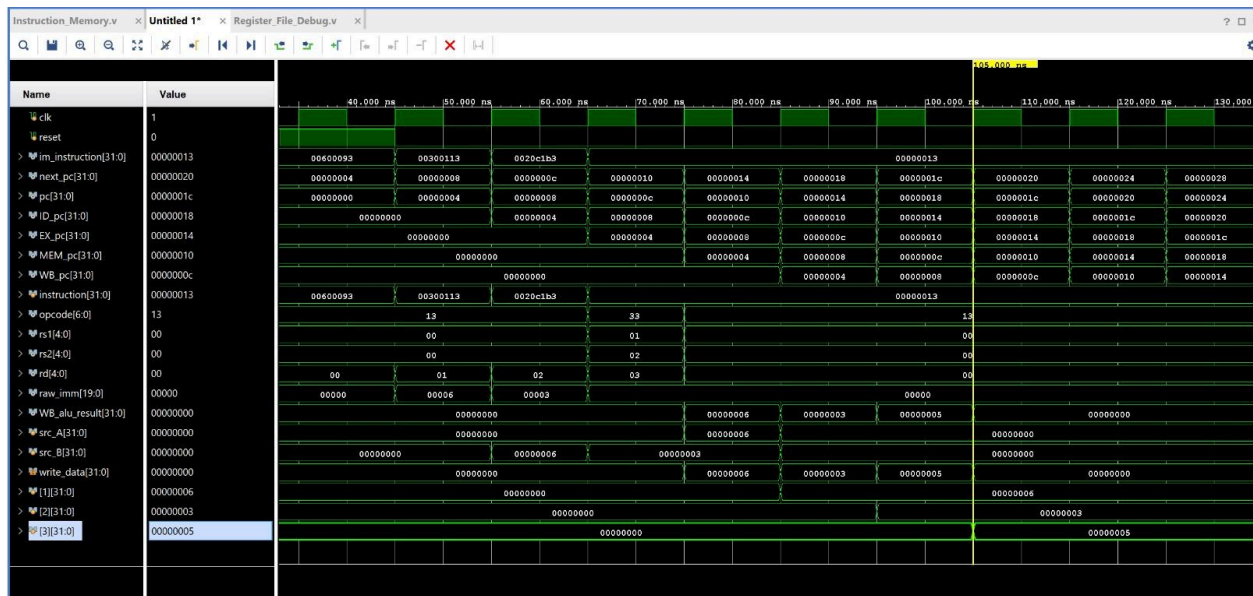
Instruction: xor
Type: R-type

34.

```
// Initialize
data[0] = {12'd6, 5'd0, `ITYPE_ADDDI, 5'd1, `OPCODE_ITYPE}; // x1 = 6 (110)
data[1] = {12'd3, 5'd0, `ITYPE_ADDDI, 5'd2, `OPCODE_ITYPE}; // x2 = 3 (011)

// XOR
data[2] = {7'b0000000, 5'd2, 5'd1, 3'b100, 5'd3, `OPCODE_RTYPE};
// xor x3, x1, x2 → x3 = 5 (101)
```

Simulation Result:

**Comments:**

The xor instruction is fetched at PC = 0x00000008 and decoded as an R-type instruction, where the processor reads the source registers x1 and x2, introducing a RAW dependency. During the execute stage, the ALU performs a bitwise XOR operation between the values in x1 and x2. The result is then written back into the destination register x3, after which the instruction retires.

Instruction: sltu
Type: R-type

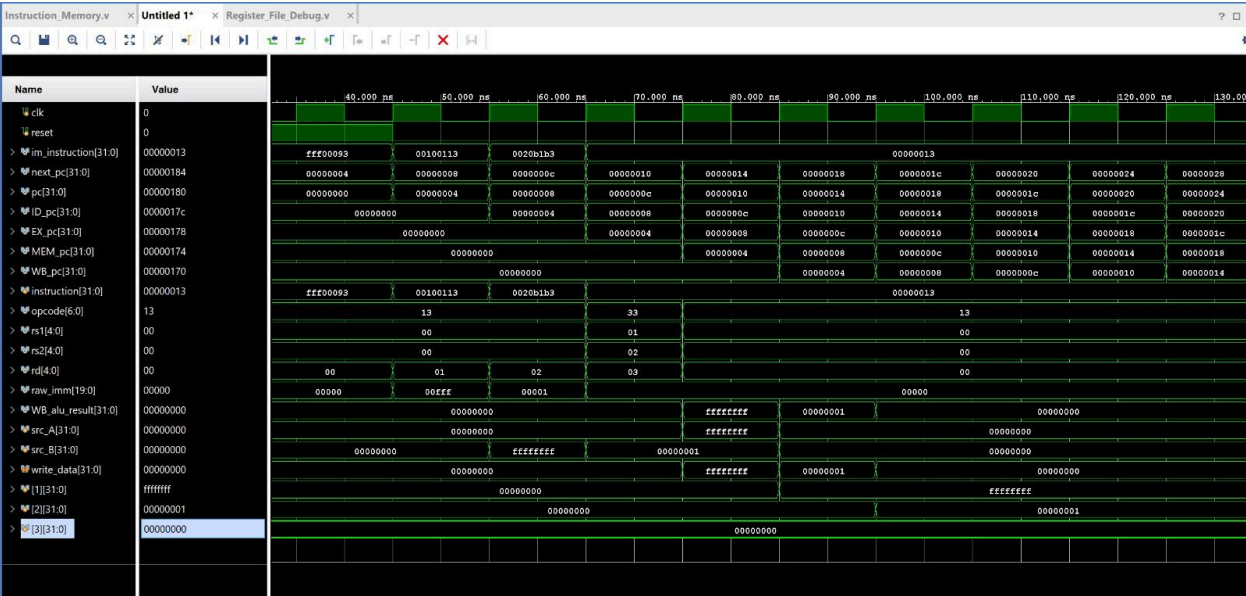
Instruction memory:

```
// x1 = -1 → 0xFFFFFFFF
data[0] = {-12'd1, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};

// x2 = 1
data[1] = {12'd1, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};

// sltu
data[2] = {7'b0000000, 5'd2, 5'd1, 3'b011, 5'd3, `OPCODE_RTYPE};
```

Simulation Result:



35.

Instruction: sra

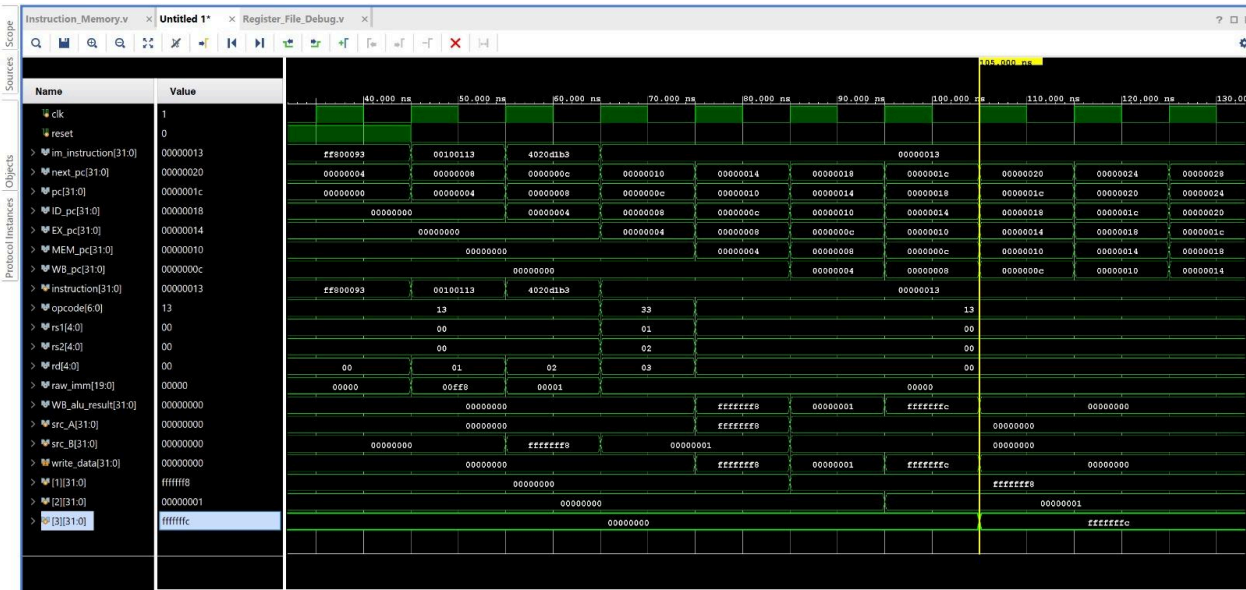
Type: R-type

Instruction memory:

```
// x1 = -8
data[0] = {-12'd8, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};
data[1] = {12'd1, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE};

data[2] = {7'b0100000, 5'd2, 5'd1, 3'b101, 5'd3, `OPCODE_RTYPE};
```

Simulation Result:



36.

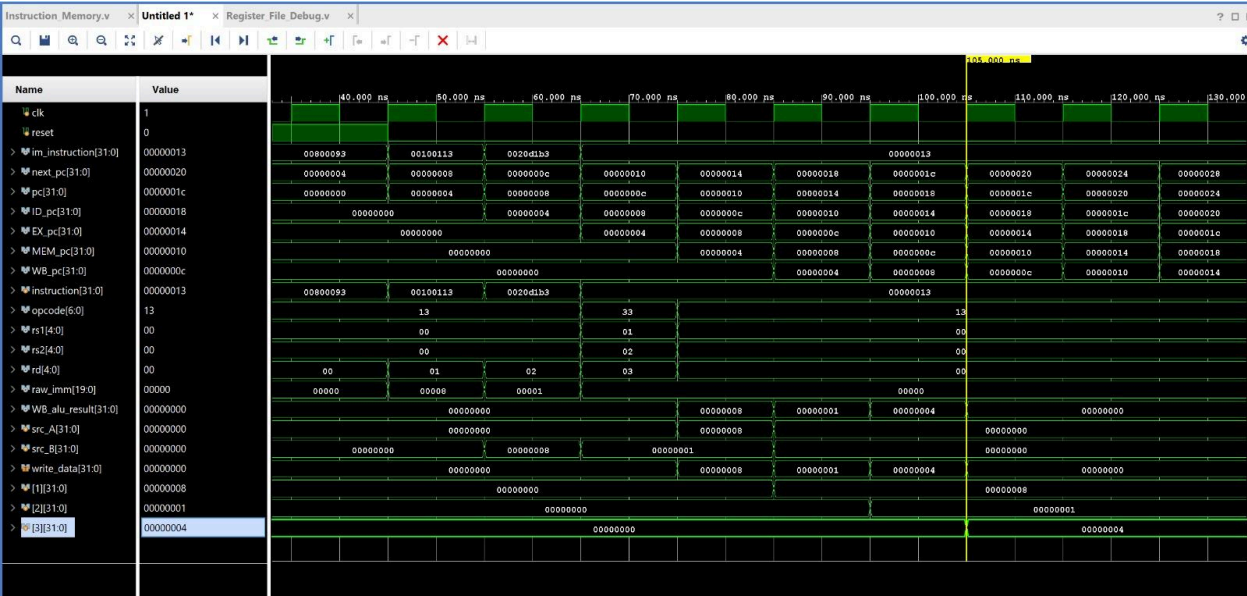
Instruction: srl
Type: R-type

Instruction memory:

```
// Initialize
data[0] = {12'd8, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 8 (1000)
data[1] = {12'd1, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE}; // x2 = 1

// Shift right
data[2] = {7'b0000000, 5'd2, 5'd1, 3'b101, 5'd3, `OPCODE_RTYPE};
// srl x3, x1, x2 → x3 = 4 (0100)
;
```

Simulation Result:



37.

Instruction: add
Type: R-type

Instruction memory:

```
// Initialize
// Initialize registers
data[0] = {12'd10, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE}; // x1 = 10
data[1] = {12'd20, 5'd0, `ITYPE_ADDI, 5'd2, `OPCODE_ITYPE}; // x2 = 20

// Perform addition
data[2] = {7'b0000000, 5'd2, 5'd1, `RTYPE_ADDSUB, 5'd3, `OPCODE_RTYPE};
// x3 = x1 + x2 = 30
```

Simulation Result:

